TILERA®

MULTICORE DEVELOPMENT ENVIRONMENT OPTIMIZATION GUIDE

DOCUMENT RELEASE 1.00
DOC. NO. UG515
MARCH 2012
TILERA CORPORATION

Publishing Information:

Document Number	UG515
Document Release Number	1.00
Date	23 March 2012

Contact Information:

Tilera Corporation
Information info@tilera.com **Web Site http://www.tilera.com**

Contents

PREFACE

About This Guide

This guide describes how to improve the performance of multicore applications that are using Tilera's Multicore Development Environment™ (MDE).

This guide applies to the MDE Release 4.0.

Intended Audience

This guide is intended for use by Tile-Gx Processor™ application developers.

Note: This guide assumes that you have already become familiar with the *Multicore Development Environment User Guide* (UG501) and *Programming the Tile Processor* (UG505).

Changes from the Previous Release

The major changes in this guide from MDE 3.0 are:

- The section on `netio-stat` has been removed, because `netio-stat` is not used in the MDE 4.0 release.
- Section 4.2 "Inlining" on page 28, Section 4.4 "Unrolling and Loop Pragmas" on page 29, and Section 4.5 "Using Intrinsics" on page 32 have been revised.
- A new section has been written: Section 2.2.2 "Data Alignment" on page 6.
- Some of the topics involving optimization in the code generation phase have been deleted.
- The former chapters "Overview of Performance on the Tile Processor" and "Optimization Tips" have been combined into one chapter. See Chapter 1: Optimizing for the Tile Processor.
- The former appendix "Optimization-Related Options for tile-gcc/g++" has been deleted.
- A new chapter describing loop and link-time optimization for the gcc compiler has been added. See Chapter 5: Loop and Link-time Optimization with the gcc Compiler.

Document Organization

This guide is organized into these chapters:

- Chapter 1: Optimizing for the Tile Processor discusses factors to consider in parallel programming.
- Chapter 2: Memory Considerations for High Performance discusses locality issues.
- Chapter 3: Performance Analysis discusses how to do performance analysis for Tile Processor applications.
- Chapter 4: Using the Optimization Features of the tile-gcc/g++ Compiler describes how to optimize a C or C++ application.

- Chapter 4: Using Feedback-Based Optimization with the tile-gcc/g++ Compiler describes a Tilera-developed method of using feedback to enhance optimization for C and C++.
- Chapter 6: Using Synchronization Methods for High Performance provides information about improving performance by using spin methods.

This guide also includes these following appendix items:

- Appendix A— TILE-Gx Performance Counter Events
- Appendix B— Suggested Reading

and an Index.

MDE Documentation

Note: `TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tilera tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own Tilera IDE Online Documentation, which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command Help → Help Contents.

- When the online help browser is displayed, select the document Tilera IDE Online Documentation in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tilera Web site at `http://www.tilera.com`.
- E-mail questions to `support@tilera.com`.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
`{ }`	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.
`[ ]`	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
`...`	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.
`$`	This is the system prompt for host-side commands.
`#`	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as `root`.
`command`	A command name appears in the Courier New font
`computer output`	Computer output appears in the Courier New font.
filename	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the Arial font.
new term	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
`TILERA_ROOT`	`TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tilera tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

CHAPTER 1 OPTIMIZING FOR THE TILE PROCESSOR

This chapter describes:

1.1 Performance Factors

Performance of an application executing on the Tile Processor™ is the product of two factors: performance per tile and the scaling of performance across multiple tiles. A high-performance application must run well on each tile as well as exploit the combined resources of many tiles. So both traditional single-core optimizations and parallel performance considerations are very important.

We recommend a two-step procedure to produce an application that achieves high performance on multiple tiles:

1. First, optimize your code to run on a single tile.
2. Then, decompose your application into several parts that can run in parallel.

If the decomposition is effective, the application will scale well onto multiple tiles. This means that on *N* tiles, performance should be close to *N* times the performance on a single tile.

1.2 Optimizing for Single-Tile Performance Improvements

Optimization of single-tile performance involves building the application, measuring performance, and modifying the application as necessary.

Single-tile performance can be broken down into making effective use of the following resources:

- CPU

This includes the choice of efficient algorithms, compiler optimizations, and special-purpose instructions, where appropriate.

- Memory system

  This includes the cache subsystem and external memory interfaces.

  Each tile has its own cache subsystem, and there is a global memory system supported by the mesh interconnect, and up to four DDR3 memory controllers that interface with external DRAM banks.

  In general, you should strive to reduce the bandwidth required from the memory system. Techniques include minimizing use of external memory, maximizing use of the local cache by keeping the *working set* of code and data small, emphasizing reuse (*temporal locality*), and accessing near-by locations (*spatial locality*).

1.3 Factors in Parallel Performance

Most high-performance embedded applications have "natural" parallelism arising from factors such as aggregating streams. Thus, there might be only a minimal need—or even no need—for explicit parallel programming in these applications.

Parallel performance arises from the following factors:

- Decomposing an application into several parts that can run at the same time.
- *Scalability*, or the effectiveness of the decomposition in spreading the application across many tiles.

In many cases, single-tile and parallel performance factors overlap. For instance, avoiding use of external memory will both speed up the single-tile case and will mitigate scalability bottlenecks for the parallel case. In rarer circumstances, there might be a trade-off. For example, a more efficient single-tile algorithm might consume more memory, which would limit parallel speedup. But this is less common, and need not be considered initially.

One basic technique is *data decomposition*, where the data on which an application works is divided into pieces that can be processed in parallel. The other basic technique is *functional decomposition*, where separate sub-functions of an application can be divided across multiple tiles.

A common case of functional decomposition is pipelining, where individual data items flow through several stages, one after the other to divide the overall work.

These two techniques can be applied in combination, such as an overall functional decomposition with individual stages further parallelized by data composition. Most real-world, high-performance applications exhibit parallelism at multiple levels, several of which need to be exploited to achieve maximum performance.

An application that exhibits perfect scalability will speed up by a factor of N when split across N tiles. In practice, the speed-up rarely achieves this limit. For instance, in a pipeline decomposition, throughput is limited by the slowest stage, so the work must be well balanced for good speed-up. In general, performance will be limited by communication costs between the decomposed pieces of the application, contention for shared resources such as locks, external memory bandwidth, and any serial or non-parallelizable portions of the algorithm (Amdahl's Law).

To achieve ideal linear speed-up, each application subcomponent would be processed entirely independently. But real applications typically need some communication across parallel subcomponents. How this communication is accomplished can be critical to scaling well. In general, you should minimize communication by organizing work so that most computation uses data that is resident on the local tile.

The above considerations are very general and apply across most application domains. For some domains, specific considerations are critical. For example:

- I/O — networking and other applications are dependent on high-speed ingress and egress of data through interfaces such as XAUI and PCIe. To achieve high performance, it might be necessary to use architecture-specific APIs to bypass the default operating system (OS) driver stack and process data in the application itself (for example, using the `gxio` API for the XAUI and Gbe high-speed Ethernet interfaces). Specific techniques are used to divide incoming packet flows among multiple worker tiles. Examples of theses techniques are simple round-robining for context-independent applications, and flow-affinity to maintain per packet-flow state on particular worker tiles.
- Specific computational domains such as video and signal processing can make effective use of special purpose instructions, including sub-word parallelism (processing four 16-bit quantities or eight 8-bit quantities with a single CPU operation), domain-specific operations such as sum-of-absolute-differences (SAD) for video encoding motion estimation, and cyclic-redundancy-check (CRC) for network check-summing and hashing.

1.4 Platform Considerations

Typically, to achieve the best performance, the specific nature of the platform must be taken into consideration, including the underlying hardware, operating system, system software libraries, and compiler used.

This section discusses some of the platform issues.

1.4.1 Chip Locality Considerations

The individual tiles are connected with a high-performance, scalable interconnect, with low-latency, high-bandwidth point-to-point connections between neighboring tiles.

The interconnect can be used both explicitly (for example, by means of TMC's user dynamic network (UDN) routines) and implicitly (for example, by means of remote cache accesses). The interconnect also connects to four high-speed external memory controllers and other high-speed I/O, but the overall bandwidth is much higher on-chip than off-chip (terabits per second as opposed to gigabits per second). So the goal is keeping communication on-chip rather than going off-chip.

For more information, see Section 2.4 "Chip Locality" on page 12.

1.4.2 Cache Locality Considerations

The Tile Processor has a large number of cores, each with its own CPU and cache subsystem. Necessarily, each core is relatively small, and in particular the caches are small and of low associativity relative to a large conventional core. (On the other hand, the aggregate size of the caches is comparable to a high-end conventional processor.)

These factors emphasize the importance of achieving a small set of code and data working set per tile, and of realizing good temporal and spatial locality. In general, the goal is keeping code and data on the tile rather than going off-tile. Locality can be optimized not only by good coding techniques, but also by using automated techniques such as *feedback-directed optimization* provided by the `tile-gcc/g++` compiler.

For more information, see Section 2.1 "Cache Locality" on page 9.

For more information about feedback-directed optimization, see Chapter 4: Using Feedback-Based Optimization with the tile-gcc/g++ Compiler.

1.4.3 Tile Locality Considerations

When using shared memory, a key concept is *coherence*, or making sure that there is only one value for each location at any point in time. Tilera's Dynamic Distributed Cache technology implements conventional multi-cache coherence, caching memory in any tile that accesses it.

Another goal is to maintain the mapping between frequently-accessed objects' (whether private or shared) home tiles, and the accessing thread or process. In practice, you typically want to lock threads or processes to tiles using *affinity*, which also prevents processes from unnecessarily sharing tiles and context-switching. (For information about the home tile, see *Programming the Tile Processor* (UG505).)

For more information, see Section 2.2 "Tile Locality" on page 10.

1.4.4 TLB Locality Considerations

Another ramification of a larger number of small cores is that virtual address translation look-aside buffers (TLBs) are relatively small. So applications referencing a wide span of code or data might benefit from using *huge pages* (also known as large pages). On the other hand, huge pages consume more physical memory, and they limit flexibility in mapping different pages with different cache properties.

For more information, see Section 2.3 "TLB Locality" on page 12.

1.4.5 Memory Balancing Considerations

The Tile Processor has up to four independent memory controllers and overall memory bandwidth is maximized if external memory accesses are spread evenly across all controllers. The operating system attempts to balance accesses by distributing physical-memory allocations across the controllers. But high-performance applications might wish to control allocation more specifically.

For more information, see Section 2.9.1 "Memory Balancing" on page 14.

1.5 Compiler Optimizations

See also Chapter 4: Using the Optimization Features of the tile-gcc/g++ Compiler.

1.5.1 Optimizing for Space Instead of Performance

Optimizing for space (by using `-Os`, which does not do inlining of functions) instead of performance can lead to better performance due to instruction cache hit rate improvements.

If instruction cache hit rates are low, then you might consider trying all variants: `-O3`, `-O2`, `-O1`, and `-Os`.

For more information, see Section 4.1 "Optimizations for Performance" on page 27.

1.5.2 Feedback-Based Optimization

The use of feedback-based optimization can result in anything from minimal gains to rather large gains, even as high as 30-50%. It is worth trying.

With feedback-based optimization, you compile your application and then run it, allowing the system to collect information as it runs. Then, you recompile the application by feeding that information back into the compiler, and the compiler tries to optimize the newly compiled application further.

For more information, see Chapter 4: Using Feedback-Based Optimization with the tile-gcc/g++ Compiler.

1.5.3-Bsymbolic Standard GNU Linker Feature

Shared library performance can be somewhat improved by linking it with the `-Bsymbolic` option. `-Bsymbolic` binds references to global symbols to the definition within the shared library, if any. Normally, it is possible for a program linked against a shared library to override the definition within the shared library, which requires an extra level of indirection on each procedure call within the shared library, unless `-Bsymbolic` is used.

1.6 Cache Optimizations

See also Chapter 2: Memory Considerations for High Performance.

1.6.1Cache Packing

Cache packing is an automated technique to enhance cache locality for code. The linker, working with profile feedback data, can re-arrange code to increase the probability that code will be resident in the cache when needed, and reduce the probability that code needed soon will be evicted from the cache.

1.6.2Cache Pinning

Try this if you see high L2 cache stalls. Some data structures are frequently accessed and you might want to pin them into the cache to avoid eviction.

1.6.3 Non-Temporal Loads and Stores

You might find that some data is used once and then discarded, as in streaming data. This data often displaces other useful data from the cache. In this case, you could use the non-temporal load/store instructions. This can be useful if your L2 cache stalls are high.

1.6.4Share Large, Read-Only Data

If you observe high L2 cache stalls, you might try keeping one copy of a large, shared static data structure. Suppose the application has a lot of read-only (static) data and the data is read from a file and processed identically on each tile. Then you should make sure you are not making multiple copies of this data to be read by each of the processes. Instead, they can all share one copy of the data placed in the L3 cache.

1.7 Memory Allocation Optimizations

See also Chapter 2: Memory Considerations for High Performance.

1.7.1Using Huge Pages

If you observe excessive performance loss due to TLB misses, then check your large and/or frequently used data structures. These might be more appropriately allocated in a huge page rather than one of the standard Linux page sizes. Many applications have small total memory footprint constraints, so `huge_pages` can tradeoff memory fragmentation for higher performance.

For more information, see the chapter on “Controlling Memory Allocation in an Application” in *Programming the Tile Processor* (UG505).

1.7.2 Using Memory Prefetching to Hide Memory Latency

If you find your application spending a lot of time in processor stalls waiting for memory fetches, then check to see if prefetching can help.

For example, assume that you have a packet-processing application. The worker tile could prefetch the next packet or two, or more, into cache while it operates on the current packet that was previously fetched. The prefetches for the next packet can also be interleaved with the processing for the current packet.

For more information, see the description of the `tmc_mem_prefetch()` function in the *Application Libraries Reference Manual* (UG527).

1.7.3 Memory Striping

Memory striping causes memory accesses to be spread across all available memory controllers automatically. It can result in very good overall memory controller balance.

For more information, see Section 2.9.2 “Memory Striping” on page 14.

1.8 Synchronization Optimizations

See also Chapter 6: Using Synchronization Methods for High Performance.

1.8.1 Changing Your Use of Locks

For some applications that are ported to Tilera® processors from other multicore processors with a small number of cores, the parallelism in the application is often limited by coarse-grain locking, or locks on large data structures. Simple locking schemes might not scale well on any multicore processor, and they can limit your application scalability to the Tile Processor, resulting in high lock contention and performance loss.

Discovering lock contention can be tricky. If you observe application behavior such as CPU stalls or low average instruction bundles per clock, lock contention might be the cause. For example, when you observe cycles per function in the profile data and you see many cycles in a “get_me_a_lock” call, lock contention might be the cause. Another example that is usually attributed to lock contention is low average instruction bundles per clock in the profile, despite having a low percentage of cache stalls.

The `lock_stat` tool is useful to track the performance of your locks. Try fine-grain locking, which adds locks to smaller parts of the large data structure instead of locking the entire data structure. For example, you would lock each row of a hash table instead of locking the entire hash table.

1.8.2 Spin Locks

Spin locks for synchronization are useful, especially for short locks and when locking is a source of overhead or stalls.

Do not enter expensive locking routines for short duration locks. The TMC library contains optimized lock implementations and different locks from which to choose. See the *Application Libraries Reference Manual* (UG527).

Do not have threads poll for a lock forever. Instead, poll for a small amount of time, then block.

1.9 Software Architecture/Application Structure

Optimizations

1.9.1 Enabling Zero Overhead Linux (ZOL) Tiles

Zero Overhead Linux (ZOL) allows you to specify a set of "dataplane" tiles, each of which will run a single, user-space task without incurring any Linux system overhead.

ZOL tiles are a subset of the Linux tiles and are useful for both threads and processes. They are useful for dataplane threads when you want to bind one thread to each tile. If you observe excessive jitter in application performance, ZOL mode can help.

Tilera recommends using ZOL mode for any application that does not tolerate latency well, such as 10 Gbps networking applications. With ZOL mode, Linux does not attempt to load-balance processes automatically onto the dataplane tiles. This protects any processes running on those tiles, but also means that load balancing must be done manually within the application.

When ZOL mode is enabled for a given tile, it is shielded from interrupts and other scheduling machinations of Linux. However, some seemingly harmless function calls actually bump you out of ZOL mode.

You can look in `/proc/tile/interrupts` for a tile-by-tile look at the interrupts you are getting. The interrupts are divided into approximately six classes. Linux timer overhead has been measured to be less than 1 percent. But more importantly, tasks will not get preempted by other tasks if they are running on a ZOL tile.

In addition, the use of ZOL tiles can affect application jitter. For example, many networking applications cannot tolerate dropped packets. Therefore, processing packets on Linux tiles (non-ZOL) could cause dropped packets due to periodic spikes in jitter. ZOL can negate this effect.

Another optimization technique you can try is the Linux `isolcpus` option. This prevents the Linux scheduler from running on a given tile.

See also Section 2.12.4 "Ameliorating Costs with the dataplane Feature" on page 17.

For more information, see the chapter on "Zero Overhead Linux Applications" in *Programming the Tile Processor* (UG505).

1.9.2 Affinitizing Processes or Threads to Cores

The Linux scheduler can cause excessive process or thread thrashing between cores for some applications, especially ones with a high ratio of threads/processes to physical cores. With Linux, you can set the affinity of a thread to a particular core (or set of cores) using the `sched_set_affinity()` function. The TMC library offers an extended set of functions to manage affinity, including the ability to work with ZOL tile sets.

For more information, see the `<tmc/cpus.h>` section of the *Application Libraries Reference Manual* (UG527).

1.10 Instruction Set Optimization: SIMD and/or Intrinsics

For single-thread computational performance, consider special-purpose instructions, notably SIMD, by means of intrinsics. Typically there is no need to drop all the way into assembly language programming. These intrinsics are particularly useful for video and wireless processing applications.

When the data values have lower precision than the full word widths (for example, 64 or 32 bits), these instructions allow you to process multiple data values in a single instruction. Special instructions that allow you to perform multiple operations on the data words in a single cycle (for example, SAD operation for motion estimation in video encoding) are also available.

For more information, see the Instruction Set Architecture (ISA) information in *TILE-Gx Instruction Set Architecture* (UG401).

1.11 Threads and Run-Time Optimizations

1.11.1 Using Thread Pools

The overhead in thread creation tends to be orders of magnitude more expensive than the overhead of switching between threads. Tilera's run-time libraries attempt to minimize the overhead in thread creation.

Some applications use a large number of threads (much larger then the number of cores), for example. In such situations, the thread creation overhead can be large.

One way to figure out if thread creation overhead is your problem is to look at the total run time and the total number of threads created by the application. Profiling should tell the number of cycles in `pthread_create()`.

Instead of creating a new thread each time you want a new thread (and incurring dynamic thread creation overhead), create a thread pool in advance. The thread pool should contain as many threads as you think you will need. You might need to experiment with the right number of threads in your thread pool. You need to pre-create enough threads that when you need to have some work done, you can simply pick one of the idling threads from your pool and give it the work to do. When a thread is done with its work, put it back on the set of idling threads. Work queue frameworks can also be useful to manage pthread pools.

1.11.2 Running Multiple Threads on a Single Core

SMP Linux handles thread scheduling. However, you might find that the overhead of thread context switching on the single-threaded Tilera CPU is high, causing cache thrashing and other phenomena. In this case, try to minimize the number of threads running per core, or increase the time between thread switching, or spread your application's threads across more cores.

1.11.3 Using Zero-Copy Buffer Management

You should use zero-copy buffer management wherever possible.

If your profile data shows excessive cycles being spent in functions where large amounts of data are copied, then you should investigate zero-copy buffer management. Applications almost always create data structures. When data needs to flow from one section of the code into another section for further processing, you should avoid doing memory-to-memory copies, which are expensive.

A best practice is to share buffers where possible. When buffers are shared, then the "downstream" process does not need to do a memory copy of the buffer, or vice versa. This can be trivial to implement, or it can be tricky (for example, if buffers cross a protection level boundary from user space to kernel space).

CHAPTER 2 MEMORY CONSIDERATIONS FOR HIGH PERFORMANCE

This chapter describes:

Note: See also the chapter on "Controlling Memory Allocation in an Application" in *Programming the Tile Processor* (UG505).

2.1 Cache Locality

A cache is a small, fast area of memory that maintains copies of recently accessed data. Caches take advantage of two common memory access properties:

- *Temporal locality* — If a given location is accessed, that location will likely be accessed again soon, so the data is retained in case it is needed again.
- *Spatial locality* — If a given location is accessed, it is likely that nearby locations will be accessed soon. For example, an application will likely access each element of an array or multiple fields of a structure.

Caches are typically structured in *cache lines*, or contiguous groups of data. The larger the cache-line size, the more data is brought in at once, which increases performance, but the more likely that unneeded data is brought in, wasting cache space and memory bandwidth.

In the Tile Processor™ each tile has its own cache subsystem, so the size of each cache is smaller than in conventional large cores. This emphasizes the importance of cache locality.

For application code, compiler optimization can affect code size, and thus performance. To a certain point, optimization will make code smaller, but beyond that, aggressive optimizations such as inlining and loop unrolling will increase size, which can reduce application performance as a

whole. Two relevant options are `-Os`, which emphasizes compact code, and `-O3`, which specifies aggressive optimization, possibly at the expense of larger code. See Chapter 4: Using the Optimization Features of the tile-gcc/g++ Compiler.

Another important cache attribute is *associativity*. In an ideal cache composed of a set of cache lines, a new line (contiguous group of data) is brought in because of a program access could be placed anywhere in the cache. But this means that looking up a location by address could result in searching the entire set of cache lines. This is very expensive, so in practice, cache lines are arranged in *sets*, such that a new line from memory can be placed in one of a small number of locations in the cache. In a *direct-mapped* cache, there is only one location in the cache for each line. In a *two-way set-associative cache*, there are two locations for each line, and so forth.

Lower-associativity caches can suffer from *conflicts*, which means that multiple memory locations being accessed close together in time might map to the same cache line. Thus, when one is accessed, it might knock another out, and so forth, resulting in excess memory traffic. Ideally, memory accesses are spread out so that they map to different cache lines over short spans of time.

In the Tile Processor, each tile has the following cache subsystem:

- A level-1 instruction cache (L1-Icache), which is a 32 KB 2-way direct-mapped cache with a 64-byte line-size.
- A level-1 data cache (L1-Dcache), which is a 32 KB 2-way set-associative cache with a 64-byte line-size.
- A level-2 unified cache (L2-cache), which is a 256 KB 8-way set-associative cache with a 64-byte line size. The set-associative cache is 4-way.

Application structure can impact locality, both for data and instructions. Instructions have natural spatial locality, because when one instruction is accessed, typically the next will be as well, except for branches.

Instructions often have temporal locality as well. Examples are a program loop or a function called repeatedly. The smaller the code, the better the locality. Ideally, the *working set*, or the code accessed frequently in any period of time, is within the L1-Icache or L2-cache size. Otherwise, instruction accesses need to go to other tiles' caches or external memory.

Instruction cache conflicts can also hurt performance. For example, assume that a frequently called function maps to the same cache lines as the calling application code. Then, every time the function is called, the application code is bumped from the cache and must be reloaded, and vice versa. The application programmer has no direct control over this at the C/C++ level, but automated tools can lay out the code to minimize the chances of cache conflict. The `tile-gcc/g++` compiler's feedback-based optimization feature lays out code to avoid cache conflicts, which can improve performance by up to 30%. For more information, see Chapter 4: Using Feedback-Based Optimization with the tile-gcc/g++ Compiler.

2.2 Tile Locality

Read-only data sections in the program executables are cached using *hash-for-home*, so that the cache lines are homed scattered across all the hashing tiles. They are cached with the `non-inclusive` attribute, so that if the L3 cache line on the home cache is evicted, the L2 cache lines on other tiles are not evicted.

The Linux kernel text (code) is handled like user text by default, using hash-for-home.

In addition, the kernel data cache homing is distributed at cache-line granularity by hash-for-home.

2.2.1 Hash-for-Home (Distributed L3 Caching)

Note: For background information and implementation details, see the chapters about memory in *Programming the Tile Processor* (UG505).

By default, the MDE uses hash-for-home (distributed L3 caching) for all except stack data. Hash-for-home allows software to define a set of CPUs to be used as a distributed L3 cache. With L3 caching, an L2 cache miss does not go to DRAM. Instead, it is directed to the home tile on the chip. If the cache line is present in the home tile's L2, then a DRAM access is avoided.

Most multithreaded applications share large amounts of data and code. When the size of frequently accessed shared memory begins to exceed the capacity of the L2 cache, distributing the memory among the caches of several tiles almost always increases performance. And tasks with larger memory requirements than others might benefit by access to remote L2 caches.

In some cases, distributed L3 caching can cause performance loss. For example, if each core has its own locally homed working set of data that fits in its L2, then distributed L3 caching could cause additional L3 access latency and thrashing as an unintended side effect.

Analyze hash-for-home and consider enabling or disabling it selectively. You can control the homing and caching of text, read-only data, data, bss, stack, heap and each mmapped page, both in user and kernel space.

As with L3 in general, hash-for-home can be enabled on a page-by-page basis. Hash-for-home reduces in-cache fragmentation across multi-tile workloads whereby one tile's cache is underused and another tile's cache is oversubscribed. Free space in neighboring tiles can be drafted into working for the tile with the full cache. Or, otherwise free space in other tiles' L2 caches is recruited into the global L3.

Also consider disabling hash-for-home in some situations. For example, for computations where you are running the same program on each of the tiles but each program is working on different datasets or input streams, your L2 stalls might be high. In this case, it is often best to use hash-for-home only for the instructions and disable it for the data. (That is, only do local allocation of the data.)

2.2.2 Dynamic Distributed Cache

Tilera's Dynamic Distributed Cache (DDC) technology caches memory locally on each tile that accesses it. The *home tile* receives all write updates. The home tile is also responsible for invalidating other caches containing stale data after a write.

To spread the load of home tile responsibilities, the Tile Processor can automatically assign a home tile from a hash or numerical function of the memory address (hash-for-home).

For more information, see the "Dynamic Distributed Cache Technology" chapter in *Programming the Tile Processor* (UG505).

2.2.3 Affinity

One implication of the cache structure is to favor *affinity*, where an application thread or process is locked to a given tile. Tile Linux will migrate private memory mappings when a process or thread migrates from one tile to another, but this is a somewhat expensive operation.

A process or thread can be locked to a tile or set of tiles using the `sched_setaffinity()` API. Shared memory mappings will not migrate, so if a shared object is mostly accessed by one thread or process, that thread or process should be locked to the tile where that shared object is homed.

The TMC™, the Tilera Multicore Components API, provides convenient operations for managing CPU affinity and memory allocation. (For more information, see the *Application Libraries Reference Manual* (UG527).)

Note: The Linux man page for the `sched_setaffinity` function implies that the set of available CPUs is contiguous starting with CPU ID 0. That is incorrect; no assumptions should be made about particular CPUs being available, or the set of CPUs being contiguous, because CPUs can be taken offline dynamically or be otherwise absent. In Tile Linux, dedicated hypervisor and bare metal environment (BME) tiles are always unavailable.

See the section on "CPU Affinity" in *Programming the Tile Processor* (UG505).

2.3 TLB Locality

Like other processors, the Tile Processor maps virtual memory to physical memory using a hardware cache called the Translation Lookaside Buffer (TLB). There are actually separate TLBs for instructions (a 32-entry ITLB) and data (a 32-entry DTLB). Each TLB entry maps a single page (for example, 64 kilobytes). Thus the total amount of memory mapped by each TLB is fairly small. TLB misses are handled by the hypervisor quite efficiently (roughly 100 cycles), but the cost can increase if a program randomly accesses large data structures.

If the hypervisor's internal data structures do not contain a TLB entry to use, it must consult the Linux page table entry instead. In this case, the time to handle a TLB miss increases to about 1,000 cycles. The hypervisor manages its TLB entries as a direct-mapped cache that holds around 1,000 TLB entries.

As in cache locality, many of the same considerations apply: smaller is better, and both temporal and spatial locality are significant. In other words, it is more efficient to reference the same or nearby data multiple times in short succession, rather than many disparate locations. (Unlike caches, conflicts are not an issue, since the TLBs are fully-associative.)

Profiling can point out excess TLB misses. For instance, using OProfile, select the `TLB_EXC` counter, where more than one event every 10,000 or so CPU cycles might indicate a problem.

Another technique to manage TLB locality involves using huge pages, where a single page (and thus TLB entry) maps, for example, 16 megabytes of memory. The dynamic memory allocation APIs provide options for allocating huge pages and advanced compiler options allow you to place static data (global variables) on huge pages as well.

2.4 Chip Locality

Whenever possible, the goal should be to keep communication and synchronization on chip. One way to ensure that is to use explicit communication techniques such as the UDN, rather than shared memory, which might spill out of the caches and into external DRAM. (Tilera's Dynamic Distributed Cache technology helps mitigate this issue by caching memory on any tile that accesses it.)

This style of application programming is called *distributed memory programming*: each component has its own local memory region, and communicates explicitly. Such applications can scale better, and be easier to analyze and tune, because the key data-sharing operations are explicit in the source code.

By contrast, a *shared memory* program will communicate by means of shared memory objects, accesses to which are indistinguishable in the program source from local object accesses. In other words, a C/C++ object reference might be to local or shared memory; you can only determine this by resolving the allocation point of the memory.

In practice, parallel applications might be hybrids of these two styles: bulk data transfer might be done explicitly, but more dynamic accesses might be more conveniently done by means of shared memory.

To determine if there are memory access bottleneck, you can use the `mcstat` and OProfile tools.

2.5 Data Alignment

In general, memory accesses are fastest if the data is *aligned*. This means, for example, that a four-byte `int` variable or field should be allocated at an address that is a multiple of 4.

The compiler and linker allocate data this way by default, but this default alignment can be overridden by "packed" attributes and other techniques. If the compiler understands that data is unaligned (for example, from use of "packed" attributes), then relatively efficient code will be produced (just a couple of extra instructions). If the compiler does not understand that data is unaligned (for example, because of casting from unaligned pointers), then a run-time trap is involved, which is relatively expensive.

In some situations, it might be beneficial to align data to stricter alignment, such as cache-line alignment. This is especially useful for frequently shared data such as lock variables, where one wishes to avoid "false sharing." (False sharing is having logically separate variables sharing a single cache line, which might induce additional coherence operations.)

A complementary case is where one wishes to allocate variables to the same cache line in order to take advantage of spatial locality, where the variables will frequently be allocated together, so that one cache line load will get both. See Section 2.1 "Cache Locality" on page 9.

2.6 Code Transformations for Data Locality

Many code transformations can affect data locality. For example, try the following techniques:

- Lay out structs so that smaller, frequently used fields are allocated together, not separated by larger fields. The `tile-gcc/g++` compiler lays out fields in memory as they are ordered in the source code.
- For multidimensional (nested) arrays, order the arrays, or loops that access them frequently, so that the final level of array (which is contiguous in memory) is accessed in the innermost loop.
- Similarly, it might be better to arrange a data structure as an array-of-structs or struct-of-arrays, depending on which order the application accesses subobjects.
- Consider using contiguous objects such as arrays and structs instead of pointer-linked structures where possible, if there are many accesses to such structures.
- Pass small objects to functions by value (a copy) rather than reference (by means of a pointer). This also makes it more likely that the compiler will hold such objects in registers rather than in memory.
- Avoid power-of-2-sized arrays that might induce cache conflicts.

2.7 Application Memory Models

Two common application memory models are:

- Private-default

  In a multiprocess environment, such as an application composed of multiple Linux processes, each process has its own private address space. Each process, even if it invokes the same program executable file as other processes, has its own copy of global variables, and addresses of global and local variables in one process are not meaningful in other processes. Shared memory

can be allocated explicitly, using `tmc_cmem_malloc()`, and so forth. The `tmc_cmem` routines provide shared memory mapped at the same virtual addresses in all processes, so such addresses can be used across processes.

- Shared-default

  In a multithread environment, such as a pthreads application, the application consists of a single process address space, with multiple separate threads of execution. There is one set of global variables shared across all threads of the process, and while there are separate thread stacks, and thus separate sets of local variables, the stacks are also in shared memory and thus addresses of all variables are meaningful across threads. Standard dynamic-memory allocation functions such as `malloc()` also allocate memory in the shared address space.

  See Section 2.10 “Memory Allocation Costs” on page 15.

2.8 User-Managed Coherence

For the finest control of caching, applications can choose to manage coherence themselves, a technique known as *user-managed coherence*. This means that memory can be allocated to be locally cacheable on all tiles. (Because Dynamic Distributed Cache technology caches locally by default, user-managed coherence is very rarely needed.)

The typical usage pattern is:

1. A single writer process will assign values to an object in shared memory, allocated with the appropriate flags.
2. When the writer is finished, it will execute a flush and memory-fence, ensuring that all updated values are written out to external memory.
3. The shared address can then be communicated to other processes, for instance by means of a pthread condition variable.
4. Reader processes will invalidate their caches, ensuring that no old values remain, and then they can access the shared object, creating copies in their own caches for efficient access.

User-managed coherence is an advanced technique and is prone to subtle, hard-to-debug errors if it is misused, for example if a semantically necessary flush or invalidate is omitted.

2.9 Memory and Memory Controllers

2.9.1 Memory Balancing

The Tile Processor has up to four independent DDR3 memory controllers, each controlling separate external DRAM devices. High-order bits in the physical address select the memory controller, and the hypervisor maps virtual addresses to physical addresses in order to balance accesses. In particular, each tile by default maps memory to the nearest controller, so users can view the chip as divided into quadrants, with each subset of 16 tiles mapping to the memory controller in its “corner”.

To see if memory balance is a potential problem, use the `mcstat` tool, which displays memory accesses per controller. For example, a memory-bound application running on relatively few tiles (just one or two quadrants) might benefit from explicitly distributing data structures onto more controllers. See Section 3.7 “Using mcstat for Memory Controller Statistics” on page 24.

2.9.2 Memory Striping

By default, the Tile Processor implements *memory striping*, or memory accesses that can be automatically distributed among the memory controllers.

Memory striping causes memory accesses to be spread across all available memory controllers automatically. It can result in very good overall memory controller balance. Check the memory controller statistics using the `mcstat` tool to see if one memory controller has much higher utilization than the others. See Section 3.7 "Using mcstat for Memory Controller Statistics" on page 24.

There are some cases where memory striping can degrade performance. The alternative to striping is to allocate memory entirely on a particular memory controller, which can be useful in some situations. If your application is spatially partitioned on different tiles in regions, then you might want to control which data structures are allocated to which memory controllers to optimize locality.

To control striping, use the Hypervisor's `stripe_memory` option. See the "Hypervisor Customization" chapter in the *Multicore Development Environment System Programmer's Guide* (UG509).

2.9.3 Using mcstat for Memory Controller Statistics

`mcstat` provides overall memory controller statistics, providing insight into both the overall amount of external memory accesses and the traffic distribution across the controllers.

For more information, see Section 3.7 "Using mcstat for Memory Controller Statistics" on page 24.

2.10 Memory Allocation Costs

The first time a page is touched, there is a cost associated with making the page ready, and with creating a suitable page table entry in the page table so that the TLB management code can find it.

If the page is already allocated on the system (for example, it is a page from an existing file that has been mapped into memory) then the first-time cost of touching the page just to get it into the page table is on the order of 10,000 cycles.

If the page must be allocated (for example, a newly-allocated anonymous page that is being added to the program's heap), it must be both allocated and zeroed, so in this case the cost is closer to 100,000 cycles.

Zeroing huge pages is correspondingly slower than zeroing small pages, so the cost of allocating and zeroing a huge page is larger, on the order of 10 million cycles.

Linux normally does not eagerly allocate anonymous pages when they are first mapped into the process's address space, but instead "lazily" allocates each one the first time the program touches it. However, in practice most applications do not benefit from this, and would rather pay the costs at allocation time. In addition, "eager" allocation means that pages of physical RAM are committed at allocation time, which allows the application to do error-checking just once, at the allocation site. The `tmc_alloc_map()` API (as well as the `malloc` and `tmc_mspace` APIs that are layered on top of it) by default do not perform any lazy allocations, so the cost of allocation is paid during the allocation API call.

When the program calls `fork()`, Linux duplicates the page table, but leaves both parent and child process sharing the same set of pages. Pages that were writable before the `fork()` are marked as read-only in both parent and child. The first process that tries to write to one of those pages will cause Linux to perform a "copy-on-write" operation, which allocates a new page and copies the contents of the old page to it. The cost of this copy is comparable to the cost of allocating a new page, as described above. The "lazy" aspect of this copy is desirable, since if the forked child calls `exec()` to start a new process, then the copying costs are never paid.

2.11 “Common Memory” Costs

The `tmc_cmem()` API can be used to manage “common memory”. Multiple processes can effectively collaborate within a region of their address space, sharing that region in a way similar to how pthreads share their entire address space. Processes can allocate or free pages within this region, and all the other processes will see the same allocations and frees, and can share pointers to data structures within the region.

The costs here are similar to the memory allocation costs described in Section 2.10 “Memory Allocation Costs” on page 15 when each process first touches a page in the “common” memory area. Pages that have already been written by another process entail paying the “allocation” cost; the first time a page is touched by any process, the full zeroing and allocation costs must be paid.

In addition, however, the normal `map` and `unmap` functions become significantly more expensive, since they must notify all the other cooperating processes of the map or unmap event, and wait for them to acknowledge. Likewise, processes must be willing to be interrupted by map and unmap requests by other processes.

For more information, see the section “Using the tmc_cmem() API to Access Common Memory” in *Programming the Tile Processor* (UG505).

2.12 “Home Cache” Management Costs

Each page in the system has a “home cache” that tells the system where the “coherence point” for the page is, or in other words, which CPU’s cache holds the canonical copy of the page’s data. (It might not be in that cache at all, of course, in which case the “canonical” data is just stored in RAM.) The “home cache” can also be hash-for-home, in which case the home of the page is distributed around the chip at cache-line granularity.

2.12.1 Changing Home Cache at Allocation Time

When pages are allocated to user space, they are always given a default home cache. Often this is the cache of the CPU that runs the allocation; it can also be “hash-for-home”, or some other home cache setting if requested by the application. If the page being allocated previously had a different home cache, the kernel has to change the homing of the page.

Changing the home of a page can add on the order of another 30,000 cycles to the page allocation costs, since the kernel must perform cache invalidation to make sure the previous “home cache” no longer has any cache data associated with the page.

In addition, the kernel might have its own kernel-space mapping for the page, and if that mapping must be updated, the kernel will also perform a kernel TLB flush to all the other CPUs, thus adding an additional cost. These flushes are done in parallel, but still increase the time necessary to change the home cache for the page.

2.12.2 Changing Home Cache at Migration Time

When a task (process or thread) moves from one CPU to another, the kernel will generally change the homing of some of the task’s pages. By default, a single-threaded process will have all of its writable pages homed on its current CPU, and when it migrates, the system will update the homing of all those pages to be on the new CPU. “Hash-for-home” pages are not migrated; neither are pages for which the user has requested homing on a specific CPU. In threaded programs, the kernel by default will only migrate the kernel and user stack pages, and any heap memory allocated by that thread.

As a general rule of thumb, it costs about 10,000 cycles per page to migrate pages when a task moves from one CPU to another.

2.12.3 Managing Non-Coherent Pages

The system treats some pages as "special" from a home cache perspective: specifically, file pages that are not being accessed by any process in a modifiable way (technically, pages that are not "shared writable"). These pages are considered "immutable" from the memory subsystem's perspective, and are cached "NC", or "non-coherently". This means that any CPU on the system is free to cache data from those pages, even if the home cache itself no longer has a copy of the data. Typically, the code of an application (the "text" of the program) is treated in this manner.

If such a page is then modified (for example, by an application creating a "shared writable" mapping for that page), Linux will flush the page from every CPU's cache and flush the TLB everywhere so that the page is treated coherently again. This is not a normal occurrence in a typical Linux system.

When a non-coherently cached page is freed (perhaps a binary was executed, then removed from the filesystem), the page is "sequestered" by Linux, since making it available for re-allocation would mean that the next task to use it would be obliged to do a cache and TLB flush interrupt on every CPU, which is generally not desirable. The set of "sequestered" pages is generally small, but if it grows in size, eventually Linux will run out of memory, and will issue a global cache flush and TLB flush and reclaim all the sequestered pages. When this occurs, a message is displayed on the console. This is not a normal occurrence in a typical Linux system.

Similarly, application code can explicitly request "incoherent" pages, if it wishes to perform explicit cache flush and invalidation handling programmatically. These pages will also have their data freely cached on multiple CPUs throughout the system. When such pages are freed, they are also "sequestered" by the kernel.

2.12.4 Ameliorating Costs with the dataplane Feature

The "dataplane" feature is useful for reducing many of the home-cache management costs. (For information, see Section 2.5.1 "Enabling Zero Overhead Linux (ZOL) Tiles" on page 8.)

One key aspect of making a CPU part of the "dataplane" involves its ability to stop its 100 Hz scheduler tick when a single process is running (and not making system calls to Linux). In this case, the small overhead of the scheduler tick is removed. (This is typically on the order of 50,000 cycles at 100 Hz, that is, less than one percent.) In addition, the TLB and cache refill costs that would otherwise be paid when the application resumed after the scheduler tick are also removed.

The kernel TLB flush operations described above are also mitigated on dataplane CPUs. Kernel TLB flushes are "deferred" when they are sent to dataplane CPUs that are running userspace code. Instead, a flag is set by the operating system for that CPU, and when the dataplane process next enters the kernel, the kernel will perform the TLB flush at that time. As a result, the running application process is shielded from the effect of global kernel TLB flushes.

Similarly, the kernel attempts to shield the process from the requirements of a cache flush when a page that used to have its home cache on the dataplane CPU is then reused with its home cache on another CPU. Normally this would require a cache flush on the dataplane CPU. However, Linux attempts to avoid reusing such pages, instead placing them on the "sequestered" list as described in Section 2.12.3 "Managing Non-Coherent Pages" on page 17.

2.13API Extensions for Managing Memory

Tilera's OS and system software libraries provide extensions of standard memory allocation and management APIs to specify the extended memory types, such as programmatically cached memory, huge pages, and so forth.

Control is also provided for memory controller placement and for advanced caching techniques, such as user-managed coherency. These APIs allocate memory dynamically, emphasizing the use of explicitly shared memory.

For more information, see the chapter on "Controlling Memory Allocation in an Application" in *Programming the Tile Processor* (UG505).

CHAPTER 3 PERFORMANCE ANALYSIS

This chapter describes:

3.1 Overview of Performance Analysis

The considerations mentioned in Chapter 1: Optimizing for the Tile Processor apply most straightforwardly to designing an application from scratch. But more typically, an existing application will be ported to the Tile Processor™. It might already be somewhat parallelized, or perhaps not. So it is important to discover and focus on the performance-sensitive portions of an application.

More broadly, achieving high performance is typically an iterative process. Even with a good overall design, it is necessary to run the application, measure its performance, discover opportunities for improvement, modify the application to achieve higher performance, and re-measure.

Traditional profiling focuses on where time is spent in the program, typically broken down by source-code routine. This is a useful way to identify key performance-sensitive algorithms or subfunctions of the application.

To achieve maximum performance, especially in parallel applications, it is also important to determine how time is being spent, or where in the machine. For example, there is little point in optimizing the code of an algorithm if the bottleneck is actually external memory latency or I/O throughput.

Parallel scalability might show up in a traditional profile (such as contention on locks or other global synchronization), or might not (for example, competition for external memory bandwidth). Tilera provides tools for detailed performance analysis, encompassing both source-code profiling and architectural metrics, using either hardware performance counters or detailed simulator modeling.

A simple technique is application-specific profiling by means of instrumenting key points and reporting time spent per operation, and so forth. Tilera® provides low-overhead and fine-granularity instrumentation tools, such as a cycle-counter API (see the `<arch/cycle.h>` section of the *Application Libraries Reference Manual* (UG527)).

3.2 Profiling on the Hardware Platform

The Tilera MDE can profile applications running on either the simulator or hardware. Data captured from either source can be viewed in the IDE's Profile view. See Section 3.4 "Using the Tilera Profile View in the IDE" on page 21.

Simulator profiling produces extremely detailed and precise information, but it is expensive because the simulator runs thousands of times slower than real hardware. Hardware profiling captures much less information, but runs at nearly full speed, and can capture data in complex real-world situations such as an application interacting with a remote data source in real time.

Hardware profiling is performed using OProfile, Tilera's customized version of the OProfile system profiler, a utility for collecting performance information. For example, you can query data and instruction cache-miss stall cycles (`MP_DATA_CACHE_STALL` and `MP_INSTRUCTION_CACHE_STALL` events). For more information, see "Profiling Applications from the Command Line with OProfile" in the "Development Tools" section of the MDE Document Index (`doc/html/profiling.html`).

See Appendix A— TILE-Gx Performance Counter Events for information about the most important performance counters and their corresponding events.

3.3 Using the perf Tool

We provide a version of the perf profiler tool for Tilera Linux. Generally, the usage is:

```
$ perf [--version] [--help] COMMAND [ ARGS ]
```

Table 3-1 lists `perf` commands.

Table 3-1. Command for the perf Tool

Command	Description
annotate	Read `perf.data` (created using the command perf record) and display annotated code.
archive	Create an archive with object files, including build-IDs found in a `perf.data` file.
bench	General framework for benchmark suites.
buildid-cache	Manage a build-ID cache.
buildid-list	List the build-IDs in a `perf.data` file.
diff	Read two `perf.data` files and display the differential profile information.
inject	A filter to augment the events stream with additional information.
kmem	A tool to trace and measure kernel memory properties.
kvm	A tool to trace/measure `kvm` Guest OS operations.
list	List all symbolic event types.

Table 3-1. Command for the perf Tool

Command	Description
lock	Analyze lock events.
probe	Define new dynamic trace-points.
record	Run a command and record its profile into a `perf.data` file.
report	Read a `perf.data` file (created using the command `perf record`) and display the profile information.
sched	A tool to trace/measure scheduler properties (latencies).
script	Read a `perf.data` file (created using the command `perf record`) and display trace output information.
stat	Run a command and gather performance counter statistics.
test	Run performance sanity tests.
timechart	A tool to visualize total system behavior during a workload.
top	System profiling tool.

An example is:

```
$ perf stat -a -A -e SNOOP_INCREMENT_READ executableFile
```

See `perf help` *`COMMAND`* for more information on a specific command.

3.4 Using the Tilera Profile View in the IDE

The Tilera Profile view is used to display and explore the content of profile data captured from applications running on the hardware or simulator platform. You can use the Tilera Profile view to examine profile files generated either within the IDE or by running an application from the command line.

For more information about using the IDE for profiling, see the chapters "Profiling an Application in the IDE" and "Viewing Profile Data in the IDE" in *Using the Tilera IDE* (UG511) or see the online Tilera IDE User Guide.

3.5 Profiling on the Simulator

The Tilera simulator generates a single XML file summarizing stall times, memory cache hits/misses, iMesh™ network traffic, I/O controller traffic, and other states. You can use `tile-eclipse --profile profile.xml` as a shortcut to display this file in the IDE.

Profiling on the simulator is recommended when you want to:

- Predict behavior on the hardware
- Collect complete statistics on the behavior of your application, including statistics that are not collectable on the actual hardware

3.5.1 Using the Profiler Control API

You can use the Profiler Control API to control the collection of profile data at run time for an application running on the simulator. This API allows profiling data to be captured around only specific code of interest. For example, you might want to exclude lengthy initialization code, or focus on specific performance-critical parts of the application.

You enable profiling by using the `void sim_profiler_enable (void)` function and disable the profiler using the `void sim_profiler_disable (void)` function.

This example shows how to profile the total overhead of a function call without including the overhead of surrounding code:

```
#include <arch/sim.h>
#include <stdio.h>

int work(int x)
{
   int i;

   for (i = 0; i < 5; i++)
   {
      x *= x;
   }

   return x;
}

int main()
{
   int i;

   // clear the profiler
   sim_profiler_disable();
   sim_profiler_clear();

   // run the work function several times, counting only the function
   // call, not the loop iteration code.
   printf("Starting work.\n");
   for (i = 0; i < 10; i++)
   {
      sim_profiler_enable();
      work(i);
      sim_profiler_disable();
   }
   printf("Finished work.\n");

   return 0;
}
```

See the *Application Libraries Reference Manual* (UG527) for more information.

3.6 Debugging Low IPC (Instructions Per Cycle)

A useful first step in performance analysis is to check how well an application is running in general. Typically you want the application to be actually executing most of the time, as opposed to waiting for memory and so forth.

An overall statistic used to measure how well an application is running is instructions per cycle (IPC). A high IPC, close to 1, indicates that most cycles are spent executing instructions, which is good. A low IPC, close to 0, indicates that most cycles are spent not executing instructions, typically waiting on cache or memory, which is typically undesirable.

The IDE Profile View calculates IPC as the executed instruction count (`MP_BUNDLE_RETIRED`) divided by the cycle count (`ONE`).

To debug low IPC, you need to figure out where in the architecture, not just where in the application, time is being spent.

Useful tools to answer this question are:

- OProfile (see Section 3.6.1 "Using OProfile to Debug Low IPC" on page 23)
- Hash-for-home (see Section 3.6.2 "Impact of Hash-for-Home on IPC" on page 24)
- `mcstat` (see Section 3.6.3 "Using mcstat to Debug Low IPC" on page 24)
- Simulator (see Section 3.6.4 "Using the Simulator to Debug Low IPC" on page 24)
- Thread/process affinity
- Compiler optimization: often `-Os` is best (a smaller size for instruction footprint), though smaller applications might benefit from `-O3` (see Section 4.1 "Optimizations for Performance" on page 27).
- Optimized library routines such as `memcpy()`

3.6.1 Using OProfile to Debug Low IPC

For information about OProfile, see Section 3.2 "Profiling on the Hardware Platform" on page 20.

Two items to look at are cache stalls and TLB misses.

For cache stalls, a good starting point is to look at these hardware counter events:

- `ONE`, to establish a cycle-count baseline
- `MP_BUNDLE_RETIRED`, to compute IPC
- `MP_DATA_CACHE_STALL`, for data cache stalls
- `MP_INSTRUCTION_CACHE_STALL`, for instruction cache stalls

If instruction cache stalls are a large component, then feedback-directed optimization and cache-packing can help substantially.

Data cache stalls require more hand analysis and optimization. A good first step is to use the IDE's Profile view to sort results based on data cache stall statistics, and see if specific functions, and thus specific data structures, stand out. (See Section 3.4 "Using the Tilera Profile View in the IDE" on page 21.) By default, statistics are aggregated per function. You can toggle a button to disaggregate and report by source line number, and/or double-click on a function to view the annotated source view for finer-grain analysis.

To examine TLB misses, use the `TLB_EXCEPTION` counter.

Note: This counts events, not cycles, so typically a lower event count is appropriate.

If exceptions per cycle seem inordinately high (for example, more than one every several thousand cycles), then there are a couple of solutions. You can use huge pages, which can be applied to static data by means of compiler options.) Or you can do the same analysis as for data-cache stalls to identify specific data structures.

3.6.2 Impact of Hash-for-Home on IPC

The data-cache-homing style used by Tilera can have a significant impact on IPC.

By default, the MDE uses hash-for-home (distributed L3 caching) for all except stack data; this tends to be the best for pthreaded shared-memory applications. But for distributed applications that do not share memory (most obviously those implemented as separate single-threaded processes), it is sometimes better to cache-home locally.

A simple experiment is to set the `LD_CACHE_HASH=ro` environment variable, which specifies to use hash-for-home for only read-only data. This can be set from the shell, or set globally for all processes by default by adding `ucache_hash=ro` to the Linux boot arguments.

3.6.3 Using mcstat to Debug Low IPC

You can look at memory bandwidth, and check that the memory controllers are not saturated, using the `mcstat` utility. Under excellent conditions, you can get up to around 50% utilization before latencies stretch out. See Section 3.7 “Using mcstat for Memory Controller Statistics” on page 24.

3.6.4 Using the Simulator to Debug Low IPC

You can use the simulator to report performance statistics, but it is slower than the other methods described above.

Cycle-level tracing can be used to debug performance issues in very fine detail, because stall causes are tracked and reported. The simulator is most readily used for standalone compute applications.

See Section 3.5 “Profiling on the Simulator” on page 21.

3.7 Using mcstat for Memory Controller Statistics

The `mcstat` command provides a convenient way to monitor memory activity.

Memory is accessed by means of multiple memory controllers. The `mcstat` command can be used to report statistics for each memory controller over a specified period of time as long as the `.hvc` file has dedicated a tile to running the `memprof` driver. The duration of this period is controlled by command-line options. Enter the command in any of the following formats:

- `mcstat -i INTERVAL_IN_SECS`

  `mcstat` repeatedly monitors intervals of the given number of seconds.

- `mcstat -w INTERVAL_IN_SECS`

  `mcstat` repeatedly monitors intervals of the given number of seconds. (Similar to `-i`, but `-w` displays more output.)

- `mcstat -t INTERVAL_IN_SECS`

  `mcstat` measures just one such interval for the given number of seconds.

- `mcstat -c command args`

  The duration is the time it takes to run the given command.

`mcstat` is most useful for checking that memory controller usage is roughly evenly balanced. If the performance of an application scales well and then flattens out, this is reason to suspect that it has saturated one or more of the memory controllers. Perhaps a more judicious use of tiles can provide a better balance.

`mcstat` has two different output formats, depending on the command line options you use. With `-i` or `-w` it uses a condensed report format that allows it to fit all the information from a single interval onto a single line. In the other two modes, it uses a line-per-memory controller to make the output easier on the eyes.

Here is an example of the line-per-controller output format:

```
    memory                                  % read   % write
controller      % peak      % read          miss      miss    latency
-------------------------------------------------------------------
         0           3          66            36        78        115
         1           1          62             5        56        112
         2           2          18            22        81        115
         3           0          17            30        41         97
```

The meaning of the colums is:

`memory controller`	Index of the memory controller, numbered clockwise starting from 0 in the top left of the chip.
`% peak`	Number of memory operations performed by the controller as a percentage of the theoretical peak. This theoretical peak is nearly impossible to achieve in practice. Even under the best conditions, the memory controller starts to be saturated (and thus a potential performance bottleneck) at 50%. Even with much lower numbers there might still be a memory bottleneck, depending on the pattern of references (see the descriptions of misses below).
`% read`	Percentage of memory operations that are reads (as opposed to writes).
`% read miss`	Percentage of read operations that page-missed in the controller as a percentage of all read operations. High miss rates result from memory reference patterns with low locality. The penalty for read misses is decreased memory bandwidth, but even with 100% miss rates, this will be about 80% of bandwidth with no misses at all. Often there is little one can do to improve miss rates.
`% write miss`	Percentage of write operations that page-missed in the controller as a percentage of all write operations. All the same considerations given above for read misses also apply to write misses, but the penalty is considerably higher.
`latency`	The average latency in core clock cycles of a load instruction that travels to the memory controller, as measured from the `memprof` driver's dedicated tile. This measurement includes cycles spent traveling across the Tile Processor's mesh networks to and from the memory controller. As a result, controllers that are farther from the `memprof`-dedicated tile will have higher latency measurements, particularly in heavily loaded systems where memory network congestion can increase routing latency.

The condensed format used with the `-i` option looks like this:

```
     mc 0       |      mc 1       |      mc 2       |      mc 3       |
 sat read  lat  | sat read  lat   | sat read  lat   | sat read  lat   |
   0   94  103  |   0   95  102   |   0   96   87   |   0   93   93   |
   0   98  105  |   0  100  105   |   0  100   87   |   0   97   88   |
   0   95  105  |   0   97  106   |   0   97   87   |   0   94   88   |
   0   98  105  |   0  100  105   |   0  100   87   |   0   97   88   |
   0   98  105  |   0  100  105   |   0  100   87   |   0   97   88   |
```

The semi-condensed format used with the `-w` option looks like this:

```
         mc 0           |          mc 1           |          mc 2           |          mc 3           |
 sat read  rm  wm  lat  | sat read  rm  wm  lat   | sat read  rm  wm  lat   | sat read  rm  wm  lat   |
   0   97   9  71  103  |   0   99   1  47  112   |   0   93   1  85   88   |   0  100   0   0   97   |
   0   96   9  77  103  |   0   99   1  58  112   |   0   94   1  85   88   |   0  100   0   7   96   |
   0   97   9  75  103  |   0   99   1  13  111   |   0   98   1  64   88   |   0  100   0  13   96   |
   0   98   6  68  103  |   0  100   0   6  112   |   0   99   1  37   88   |   0  100   0   0   96   |
   0   97  11  72  103  |   0   99   1  26  112   |   0   97   1  40   88   |   0  100   0   0   96   |
```

In these formats, there are up to five columns for each controller, and the correspondence to the less compressed format is as follows:

`sat`	% peak (percent of theoretical peak throughput)
`read`	% read (percent of read operations, as opposed to write operations)
`rm`	% read miss (percent of read operations that miss an open bank)
`wm`	% write miss (percent of write operations that miss an open bank)
`lat`	latency (average load instruction latency)

See also the `memprof` command.

3.8 Using the Simulator Control API to Analyze Performance

The Simulator Control API allows an application to enable or disable tracing at run time. Enabling tracing around performance-critical code shows detailed cycle-by-cycle information about what the code is doing, such as instructions being executed and the like. This is particularly useful to identify why and where the program is stalled.

Simulator tracing can be enabled either on the command line or programmatically.

For example, an application can enable or disable instruction tracing while executing by using the `void sim_set_tracing( int32_t` *`mask`*`)` function.

You can specify that tracing should be turned off (by using the `SIM_TRACE_NONE` parameter), or use the bitwise OR of various flags.

For additional information about trace functions, see Section 3.9 “Using the Trace Viewer” on page 28.

3.8.1 Example of How to Obtain a Trace

This example shows how to obtain a trace containing only the instructions within an application’s inner loop:

```
#include <arch/sim.h>
#include <stdio.h>

int main()
{
   int i;
   volatile int sum = 0; // keep the compiler from eliminating our inner loop

   printf("Preparing to trace\n");
   sim_set_tracing(SIM_TRACE_CYCLES |
                   SIM_TRACE_DISASM |
                   SIM_TRACE_LINES |
                   SIM_TRACE_STALL_INFO);

   sum = 0;
   for (i = 0; i < 3; i++)
   {
      sum += i;
   }

   sim_set_tracing(SIM_TRACE_NONE);
   printf("Finished tracing\n");

   return 0;
}
```

See the *Application Libraries Reference Manual* (UG527) for more information.

3.9 Using the Trace Viewer

The `tile-trace-view` command enables users to access the trace viewer, a graphical viewer for traces produced by the simulator. These traces provide information on what each tile is doing, such as executing, or stalling on a named resource. Trace views provide data on the following:

- Aggregated statistics file output to a `prefix.xml` file
- Network trace files output to a `prefix.tile*.router` file
- Tile processor trace files output to a `prefix.tile*.mainproc` file

When you run `tile-trace-view`, you can pass as arguments the list of trace files you want the program to process and summarize. For example:

```
$ tile-monitor --simulator --trace-tile-view ttv other-arguments ...
$ tile-trace-view ttv.*
```

You can also select the list of trace files from within the trace viewer.

Three views or visualizations are provided: a Trace Chart, Scatter Plot, and Network Animation.

For more information, see the chapter on "Viewing Tile Processor Architecture Traces Using tile-trace-view" in the *Multicore Development Environment User Guide* (UG501).

CHAPTER 4 USING FEEDBACK-BASED OPTIMIZATION WITH THE TILE-GCC/G++ COMPILER

This chapter describes Tilera's implementation of feedback-based optimization (FBO) used with the `tile-gcc/g++` compiler:

See also Chapter 4: Using the Optimization Features of the tile-gcc/g++ Compiler.

4.1 Overview of Feedback-Based Optimization

`tile-gcc/g++`'s optimizer makes heuristic guesses about the code it is compiling, such as how frequently an `if` predicate will be true and how many times a given loop will execute. The performance of the resulting program depends in part on the quality of those guesses.

FBO is an advanced technique for replacing the guesswork with actual data collected by running a specially-instrumented executable. This requires that you compile the program twice, first to create an instrumented executable and again to use the information collected by that executable.

With the data produced by FBO, the compiler will know:

- How often loops are executed, which can be used to guide unrolling and other loop optimizations
- Which blocks of code are executed more often, which can be used to guide code scheduling, layout, and register allocation decisions

Note that feedback data is only used as a hint for optimization, not as a guarantee for what future program inputs must look like. The compiler will not make assumptions that produce incorrect code for inputs different from the training set.

In addition to generating better code, FBO also replaces the essentially arbitrary layout of functions in memory with one tuned to reduce cache conflicts.

FBO can often speed up a program by 10% to 30%.

4.2 Example: How to Specify Feedback-Based

Optimization

The steps in this example are described more fully in Section 4.3 “How to Perform Feedback-Based Optimization” on page 30.

1. Compile instrumented object files:

```
$ tile-gcc -O2 -ffeedback-generate -c *.c
```

2. Link instrumented object files:

```
$ tile-gcc -O2 -ffeedback-emit=/tmp/raw_feedback -static *.o -o my_app
```

3. Discard old feedback data from previous runs:

```
$ rm -rf *.gcda raw_feedback
```

4. Run the instrumented executable on a “training set”:

```
$ tile-monitor --pci --batch-mode --here --upload my_app /my_app \
  --run - /my_app my_arg1 my_arg2 - --download /tmp/raw_feedback raw_feedback
```

   To use the simulator instead, replace `--pci` with `--image tile64 --functional`.

5. Convert and combine all the raw feedback data into a single file:

```
$ tile-convert-feedback -o my_app.feedback raw_feedback
```

6. Compile feedback-optimized objects:

```
$ tile-gcc -O2 -ffeedback-use=my_app.feedback \
  -freorder-blocks-and-partition -c *.c
```

7. Link feedback-optimized objects:

```
$ tile-gcc -O2 -ffeedback-use=my_app.feedback -static *.o -o my_optimized_app
```

4.3 How to Perform Feedback-Based Optimization

Figure 4-1 shows the workflow of the FBO tasks, which are explained in the following sections.

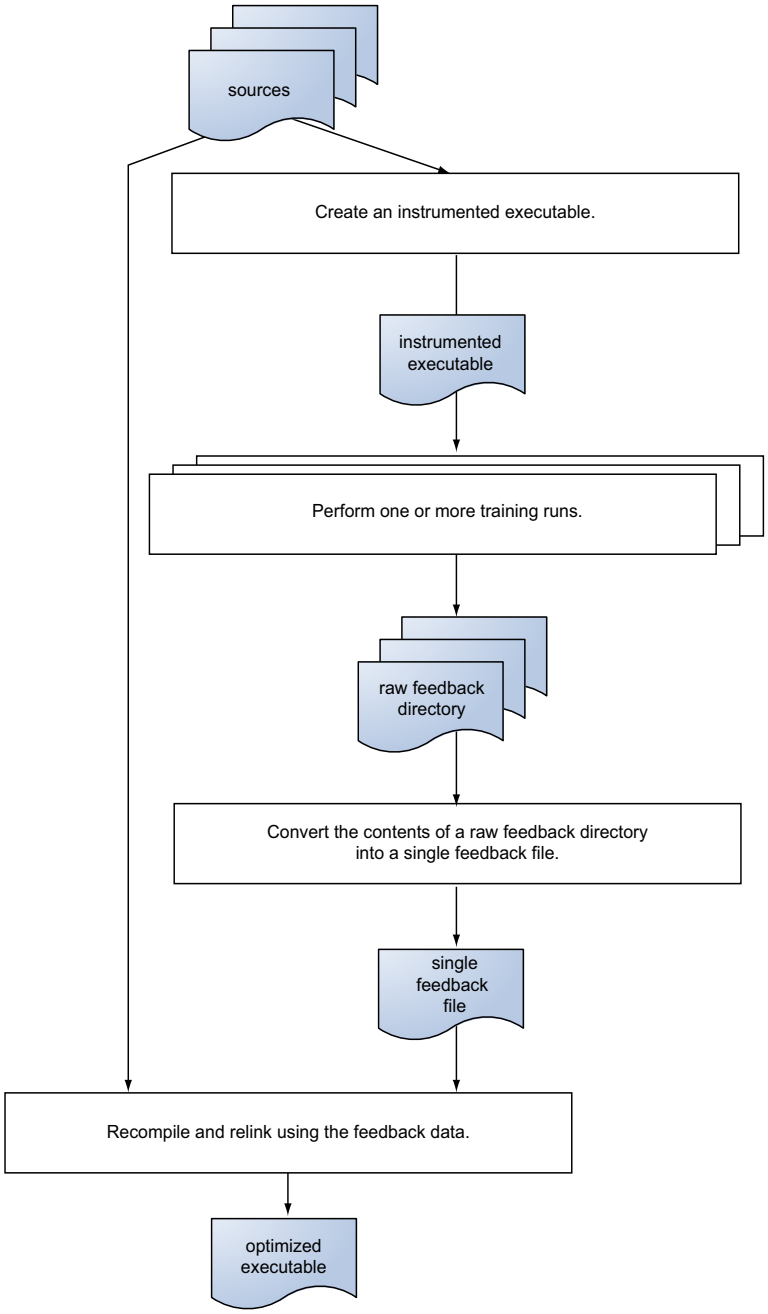

Figure 4-1: Feedback-Based Optimization Workflow

4.3.1 Creating an Instrumented Executable

To create an instrumented executable:

1. Compile using `-ffeedback-generate`, which will create instrumented code that collects feedback data.

2. Link with `-ffeedback-emit=`*`directory`* to indicate where to write the collected data. Libraries should typically be linked without specifying `-ffeedback-emit`, so that each specific executable linked against that library can make a different choice as to where to emit the feedback data.

You must use the same optimization level, such as `-O2`, for both the instrumented binary and the final optimized executable.

Instrumented executables collect two separate kinds of feedback data: data used by the compilation step, and data used by the linking step.

The standard option, `-ffeedback-generate`, collects both compiler and linker data. `-ffeedback-generate-compiler` collects only compiler data, and `-ffeedback-generate-linker` collects only linker data. Usually you want to collect both kinds of feedback data, but there are exceptions; for example, the MDE provides feedback variants of its static libraries that only collect linker data. Collecting compiler data would not be helpful because the library is not being recompiled, but linker data helps linker-optimized executables place those library functions in memory locations that reduce cache misses.

4.3.2 Performing Training Runs on the Instrumented Executable

You need to perform one or more training runs on the instrumented executable with representative options and test inputs, or a "training set." The purpose is to collect information about the program's typical behavior. The training sets become part of the `-ffeedback-emit` output, which the compiler uses to optimize the program.

For example, if a function is not executed by any training set, the compiler will have no information to optimize it and might choose to move it into a special "cold" section containing code not expected to be run.

4.3.3 Converting Raw Feedback into a Single Feedback File

The `tile-convert-feedback` tool converts the contents of a raw feedback directory, which might contain data from many different processes, into a single feedback file suitable for use with `-ffeedback-use`. For example:

```
$ tile-convert-feedback directory -o filename
```

Each instrumented process records its raw feedback data in a uniquely-named memory-mapped file in the directory specified by `-ffeedback-emit`.

To help in applying FBO to programs that never exit, each instrumented process continually updates its output file while it runs. This allows raw feedback files to be copied at any time, including while the program is running or after it crashes.

It is common to create many raw feedback files while collecting feedback data, either because the application consists of multiple processes or because the developer runs several training runs to collect more data.

Because each raw feedback file can be fairly large, and because working with dozens or hundreds of them would be awkward, `tile-convert-feedback` combines an arbitrary number of raw files into a single smaller file suitable to feed the `-ffeedback-use` step where the program is recompiled. This conversion step is required, even if only a single raw feedback file is produced.

Note that the precise layout of a raw feedback directory is considered opaque and subject to change. It is up to `tile-convert-feedback` to interpret and process the directory's contents, which is why it takes the directory name as an argument rather than individual files within it.

4.3.4 Recompiling and Relinking Using the Feedback Data

The final step is to recompile and relink with `-ffeedback-use=`*`file`*, which will optimize your executable using the feedback file generated previously. Instead of `-ffeedback-use=`*`file`*, you can specify `-ffeedback-use-compiler=`*`file`* to do compiler feedback optimizations only, and `-ffeedback-use-linker=`*`file`* to do linker feedback optimizations only.

You must use the same optimization level, such as `-O2`, for both the instrumented binary and the final optimized executable.

4.4 Compiler/Linker Options Useful in Improving Performance

These options are not necessary for FBO, but are recommended since they will usually improve performance:

- `-freorder-blocks-and-partition` will separate each function into "hot" and "cold" portions. Instruction memory locality is improved if all the hot portions are collected together. Cold code is that which is not expected to be executed at run time, while hot code is believed to be useful.
- `-static` is especially useful for FBO because `tile-gcc/g++` can only select cache-friendly addresses for those functions statically linked into the executable. Linking with the `-static` option gives the link optimizer freedom to choose optimized addresses for system library code, which can further improve performance.

  Functions in a shared library, such as `libc.so`, have their addresses fixed ahead of time, so `tile-gcc/g++` is unable to optimize them. This reduces the effectiveness of FBO.

4.5 How Feedback-Based Optimization Improves Performance

4.5.1 Compiler Feedback

Compiler feedback improves code generation by recording various facts, such as branch probabilities and loop execution counts. This process helps the compiler rearrange code to "straighten out" likely code paths to use the caches better. This data is collected by incrementing static counters "inline" in the code, and is therefore quite efficient.

4.5.2 Linker Feedback

Linker feedback improves performance by selecting addresses for functions that reduce cache conflicts. Each instrumented function calls into a runtime library that tracks the order in which functions execute.

For example, suppose the program executes function A, then function B, and then A again (as when A calls B). The feedback mechanism recognizes that if A and B alias in the cache, then B will evict A, causing cache misses when A is executed again. If non-aliasing addresses are selected for these functions, both can coexist in the cache at the same time.

Because collecting linker feedback requires calling into a run-time library on each function call or return, it slows down the instrumented executable more than (the process of) collecting compiler feedback does.

Most standard system static libraries are shipped with special linker feedback-collecting variants that are automatically selected by the compiler when appropriate. These libraries collect linker data for system library code so that their functions can be efficiently interleaved with the functions in the calling executable. But the instrumented system libraries do not bother collecting compiler feedback because they are not going to be compiled again, just linked.

4.6 Stale Data in Feedback Files

It is often convenient to create a feedback file once, and then use it repeatedly during development. When code is modified after feedback data is collected, the feedback data might no longer apply to it.

Fortunately, the data will degrade gracefully and still be applicable to unmodified functions. It does not need to be regenerated after each change in order to provide some value.

A feedback file contains a checksum for each function (based on the function's internal representation in the compiler, rather than directly on its program text). When the checksum for feedback data no longer matches a modified function, `tile-gcc/g++` emits a warning and ignores the feedback data for that function.

Linker feedback data also can become increasingly incorrect as new functions are added and function call patterns change, but will still attempt to use it. `tile-gcc/g++` does not pay any attention to checksums for linker feedback, but simply optimistically assumes that its old data is still valid.

CHAPTER 5 LOOP AND LINK-TIME OPTIMIZATION WITH THE GCC COMPILER

This chapter describes Tilera's implementation of loop-based, automatic vectorization, automatic parallelization and link-time optimization used with the `gcc` compiler:

See also Chapter 4: Using Feedback-Based Optimization with the tile-gcc/g++ Compiler.

5.1 Overview of Loop Optimization

Note: On the Linux x86 host machine, the `tile-gcc` command invokes the `gcc` 4.4 compiler. On the Native (Tile) side, the `gcc` command invokes the gcc 4.4 compiler while the `gcc46` command invokes the gcc 4.6 compiler.

Note: The `gcc` 4.6 compiler is only available in native mode (on the Tile side).

Loop blocking and loop parallelization work on very simple, restricted examples, on both `gcc` and `gcc46`.

Loop interchange, *loop distribution*, and *strip mining* only works on `gcc46`. Only loop interchange is useful.

Auto-prefetching works on both `gcc` and `gcc46`, with `gcc` generating L3 pre-fetches.

In general, using loop interchange, loop distribution, strip mining and Auto-prefetching have a lot of limitations; those optimization methods do not seem mature enough for real programs, due to the following issues:

- There are major limitations on what code blocking can handle. Loop optimizations fail to block matrix multiply operations due to loop-carried dependencies.
- Block size is insensitive to cache parameters or access patterns.
- Code bloat can occur.
- You may experience slow compile times.

Vectorization works on `gcc` if there is alignment information, but it can work on `gcc46` even without alignment information. However, the benefit of vectorization is drastically reduced if alignment information is not available.

Link-time optimization is a feature available on `gcc46`. It is relatively simple to use and can yield a modest improvement in performance.

You can use gcc (either version 4.4 or 4.6) to perform loop optimizations to improve locality and cache utilization. Relevant optimizations include loop blocking, loop interchange, loop distribution, and strip-mining (see gcc man pages for more info). These optimizations are disabled by default. To enable these optimizations, specify the corresponding compiler flags for the gcc compiler, as shown in Table 5-1.

Table 5-1. Loop Optimization Compiler Flags

Method	Compiler Flag	gcc Version Support
Loop Blocking	`-floop-block`	`gcc, gcc46`
Set loop blocking size	`--param loop-block-tile-size=N`	`gcc46`
Loop interchange	`-floop-interchange`	`gcc, gcc46`
Loop strip-mining	`-floop-strip-mine`	`gcc, gcc46`
Loop distribution	`-ftree-loop-distribution`	`gcc46`
Loop array pre-fecthing	`-fprefetch-loop-arrays`	`gcc, gcc46`
Loop parallization	`-ftree-parallelize-loops=`*N*	`gcc, gcc46`
Link-time loop optimization	`-lto`	`gcc46`

5.2 Using Loop Optimizations

These optimizations work with simple examples and yield some reasonable performance gains. Some examples of optimized code are available in gcc man pages.

For example, consider the following code:

```
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    a[i][j] = b[j][i] * c[i][j];
```

gcc blocks this code into 51 by 51 blocks, allowing strips of `a[]` and `c[]` arrays to be reused without being evicted between consecutive iterations of the loop.

Here is the performance of the loop with and without blocking, for N=16384:

	Compile-time (secs)	Run-time (secs)	Size (bytes)
no blocking:	0.2	203	2046
blocking:	5.0	25	2310

Thus loop optimization provides an 8x performance gain, with 13% code bloat. Note that the compilation overhead with blocking is significant, even for such a simple loop.

In practice, loop blocking appears to be able to handle a very limited set of loops. The above optimization works fine if a, b, and c are arrays. If they are pointers passed as arguments into a function, the optimization would fail even if you specify the pointers point to distinct object, by marking the pointers as *restrict*. Furthermore, the optimization fails to block matrix multiply due to its loop carried dependence, even though there are existing techniques to handle it.

When blocking, finding the optimal blocking size is a non-trivial problem, which requires analyzing the accesses in the loop nest, knowledge of all levels of the cache hierarchy, TLB size, etc. `gcc` currently does not do any such analysis, and instead uses a default blocking size of 51 iterations. You can only change this parameter manually using the flag: `--param loop-block-tile-size=N`.

Note: The flag `--param loop-block-tile-size=`*`N`* only works with `gcc46`.

`gcc` includes parameters on the L1 cache size, L2 cache size, and L1 cache line size, but they have no effect on the output of the compiler in these examples.

Loop interchange is relatively simple optimization that can provide big benefit, but it is also easily done by hand.

```
for (j = 0; j < N; j++)
  for (i = 0; i < N; i++)
    A[i][j] = A[i][j] * c;
```

This loop runs significantly faster if we interchange the two loops, to improve spatial locality. This optimization works on `gcc46`, but not `gcc`.

Loop distribution is a feature that is available on `gcc46`. You can give `gcc` a loop and specify at command line that you want the compiler to distribute the loop. But there does not appear to be any intelligence on when it should be applied, or when or why it's profitable.

Strip-mining works on `gcc46` but not `gcc`. Historically it is a technique used to enable vectorization. However, on `gcc` strip-mining is a requirement for vectorization, and I see no benefit for performing strip mining at all. Auto-prefetching works on both `gcc` and `gcc46`. To enable it, use the `-fprefetch-loop-arrays` flag, for which `gcc` generates L3 pre-fetches. You can tune the output of auto-prefetching using the `--param prefetch-latency=`*`N`*, and `--param simultaneous-prefetches=`*`N`* options.

5.3 Using Automatic Vectorization

You can use gcc to perform automatic vectorization on code.

Here is a loop example:

```
char a[N], b[N], c[N];
int main()
{
  int i, j;

  for (i = 0; i < N; i++)
    a[i] = b[i] + c[i];

  return a[0];
}
```

Without vectorization or unrolling, `gcc` generates the following inner-loop:

```
ld1s: r14, r10; addi: r10, r10, 1
ld1s: r15, r12; cmpeq: r13, r10, r16; addi: r12, r12, 1
addx: r14, r15, r14
st1: r11, r14; addi: r11, r11, 1
beqzt: r13, .L2
```

Note there is a bubble after the second load, so it takes six cycles to do one iteration on cache hits. With vectorization, `gcc` generates:

```
ld: r13, r12; addi: r12, r12, 8
ld: r14, r11; addi: r11, r11, 8
v1add: r13, r14, r13
st: r10, r13; addi: r10, r10, 8
cmpeq: r13, r10, r15
beqzt: r13, .L2
```

Accounting for the bubble, it takes seven cycles to do 8 iterations of work. Speedup on cache hits is 8 * 6 / 7 = 6.86. The measured speedup for N=1000000000 is: 4.14.

This example works on both `gcc` and `gcc46`, because the inputs are global arrays with known alignment information. Without alignment information, `gcc` needs to generate code to handle unaligned loads or stores, and only `gcc46` is able to do so.

Consider this example:

```
char A[N], B[N], C[N];

int __attribute__((noinline))
  foo(char* __restrict__ a,
      char* __restrict__ b,
      char* __restrict__ c)
{
  int i, j;

  for (i = 0; i < N; i++)
    a[i] = b[i] + c[i];

  return a[0];
}

int main()
{
  return foo(A, B, C);
}
```

`gcc` does not vectorize this code at all. `gcc46` vectorizes the code, but incurs overhead for handling the unaligned loads (using `dblalign`) and unaligned stores (using byte stores). The overall speedup for this loop is only 2.5.

5.4 Using Automatic Parallelization

`gcc` can perform automatic parallelization of loops that are inherently parallel. It uses the same engine that drives the loop blocking optimization, so the two optimization share the same limitations listed previously. To enable parallelization, use the option `-ftree-parallelize-loops=`*N*, where N is the desired number of threads. If it succeeds, `gcc` will generate OpenMP directives to parallelize the loops.

You can combine loop blocking with loop parallelization.

5.5 Using Link-time Optimization

Link-time optimization, also known as whole-program optimization, is a new feature first made available in `gcc` 4.5. It is enabled with the compiler flag `-flto`. When enabled, the compiler is run at link-time, when information about the whole program is available to the compiler, thus allowing it to make better optimization decisions. Some optimizations that can be performed better within this frame work include in-lining and inter-procedural constant propagation.

Even though the performance gain is often modest, it notes that program size often decreases drastically. Unlike loop optimizations, link-time optimization seems relatively robust and ready for use with real applications, and has some tangible gains– albeit not necessarily performance ones. The overall gain is about 2%. On average link-time optimization reduces executable size by 14%.

Note: The `-fwhole-program` flag did not make any difference in either performance or executable size.

CHAPTER 6 USING SYNCHRONIZATION METHODS FOR HIGH PERFORMANCE

This chapter describes how to use synchronization methods for high performance:

- Section 6.1 Spin Methods to Improve Performance

For more information, see the "Performance Tuning" and "Spinning Shared Memory Synchronization" sections in the "Tilera Multicore Components (TMC) Library" chapter of the *Application Libraries Reference Manual* (UG527).

6.1 Spin Methods to Improve Performance

Users with applications that have significant locking are encouraged to use the operations described in this section. Couple that effort with fine-tuning methods to tune spinning parameters, and the end result can be a large reduction in the total wall clock time and in-kernel "system" time.

Two API functions, `pthread_mutex_lock()` and `tmc_sync_mutex_lock()`, cause example applications to spin for a little while before calling into the kernel's futex support to queue up and go to sleep, pending a lock release.

You can achieve better performance results by linking statically, which avoids some of the overheads caused by branching between shared objects, taking more TLB misses and perhaps icache misses, and the like. From the use of -static, the number of cycles drops when using `pthread_mutex_lock()` from about 275 to about 180. Likewise, the use of `atomic_increment()` and `tmc_sync_mutex_lock()` drops the number of cycles from around 180 cycles to around 100 cycles. And using `tmc_spin_queued_mutex_lock()` brings the number of cycles needed down from 80 cycles to about 70 cycles.

In tests the code was modified to run the example once, then run it again to derive the lower number of cycles (note that you have to build your application using –pthread). See the results below.

New Operation	Cycles
`atomic_increment`	181
`tmc_spin_mutex_lock`	22
`tmc_spin_mutex_unlock`	21
`tmc_spin_queued_mutex_lock`	81
`tmc_spin_queued_mutex_unlock`	22
`tmc_sync_mutex_lock`	153
`tmc_sync_mutex_unlock`	144

pthread_mutex_lock	275
pthread_mutex_unlock	254

There are a number of factors that contribute to the number of "outliers", those operations that use more than 50 cycles:

- The atomic_ increment() operation must fake a 32-bit atomic operation by hashing the Virtual Address (VA) of the address to get a suitable lock; acquiring the lock via tns; doing the read-modify-write operation; then doing a memory fence, and releasing the lock.
- The tmc_spin_queued_mutex_lock() operation provides a fairness guarantee, which is important in some contexts and must work harder to make it true (see the comments in the Tilera Multicore Components (TMC) library header file: tmc/spin.h).
- The tmc_sync_mutex_lock() and tmc_sync_mutex_unlock() routines are implemented with the same mechanism as the atomic_increment() routine.
- The pthread_mutex_lock() and pthread_mutex_unlock() routines are also implemented by means of atomic_increment()-type mechanisms, and is a bit slower because it must use more overhead to maintain the pthread semantics.

6.1.1 Library Functions Relating to Spinning

- atomic_increment (*var*)

  Atomically increment a location in memory.

- cmpxchg(var, old, new)

  The content of the variable *var* will be replaced with new if the current value is old. Regardless, the current value of *var* before the operation is returned.

- tmc_spin_mutex_lock()

  Lock a mutex.

- tmc_spin_mutex_unlock()

  Release an already held mutex.

- tmc_spin_queued_mutex_lock()

  Lock a queued mutex.

- tmc_spin_queued_mutex_unlock()

  Release an already held queued mutex.

- tmc_sync_mutex_lock()

  Lock a mutex.

- tmc_sync_mutex_unlock()

  Release an already held mutex.

- pthread_mutex_lock()
- pthread_mutex_unlock()

APPENDIX A—TILE-GX PERFORMANCE COUNTER EVENTS

This appendix describes the most important performance counters on the TILE-Gx processor family:

- Section A.1 Process Statistics Events
- Section A.2 Stall Statistics Events
- Section A.3 Memory Cache Statistics Events
- Section A.4 iMesh Network Traffic Statistics Events

These statistics all have a minimum count of 1,000 and a default count of 800,000.

A.1 Process Statistics Events

Table 1. Process Statistics Events

Hardware Event	IDE Profile View Statistic	Description
ONE	Cycle Samples	Number of samples, taken every *N* clock cycles.
	Cycles (Estimated)	Estimated clock cycles, based on samples taken.
INSTRUCTION_BUNDLE	Instruction Bundles	The event occurs when a valid instruction bundle is committed.
	Bundles Executed Per Cycle	Instruction bundles retired, relative to number of cycles.
INTERRUPT	Interrupts	The event occurs when any interrupt is taken.
	Interrupts Per Cycle	Interrupts, relative to number of cycles.
PASS_WRITTEN	Pass SPR Writes	Pass SPR writes.
	Pass SPR Writes Per Cycle	Pass SPR writes, relative to number of cycles.
FAIL_WRITTEN	Fail SPR Write	Fail SPR writes.
	FAIL SPR Writes Per Cycle	Fail SPR writes, relative to number of cycles.
DONE_WRITTEN	Done SPR Write	DONE SPR writes.
	Done SPR Writes Per Cycle	DONE SPR writes, relative to number of cycles.
WATCH	Watch SPR	The event occurs when the Watch SPR detects an address or address range.
	Watch SPR Ratio	Watch SPR events, relative to number of cycles.

A.2 Stall Statistics Events

Table 2. Stall Statistics Events

Hardware Event	IDE Profile View Statistic	Description
INSTRUCTION_STALL	Instruction Stalls	The event occurs when a valid instruction in pipeline Decode stage stalls for any reason.
	Instruction Stalls Per Cycle	Instruction stalls, relative to number of cycles.
L1D_FILL_STALL	L1 Data Fill Stalls	The event occurs when a memory operation stalls due to L1 Dcache being busy doing a fill.
	L1 Data Fill Stalls Per Cycle	L1 data fill stalls, relative to number of cycles.
LOAD_HIT_STALL	Load Hit Stalls	The event occurs when an instruction 2 cycles after a load stalls due to a source operand being the destination. The L1 DCache hit latency is two cycles, so the instruction would stall on a miss but not on a hit.
	Load Hit Stalls Per Cycle	Load hit stalls, relative to number of cycles.
LOAD_STALL	Load Stalls	The event occurs when an instruction stalls due to a source operand being the destination of a load instruction. This event happens on all cycles that stall except for the one that is two cycles after the load, which is counted by the LOAD_HIT_STALL event.
	Load Stalls Per Cycle	Load stall, relative to number of cycles.
ALU_SRC_STALL	ALU Stalls	The event occurs when an instruction stalls due to a source operand being the destination of an ALU instruction.
	ALU Stalls Per Cycle	ALU stalls, relative to number of cycles.
IDN_SRC_STALL	IDN Stalls	The event occurs when an instruction stalls due to the IDN source register not being available.
	IDN Stalls Per Cycle	IDN stalls, relative to number of cycles.
UDN_SRC_STALL	UDN Stalls	The event occurs when an instruction stalls due to the UDN source register not available.
	UDN Stalls Per Cycle	UDN stalls, relative to number of cycles.
MF_STALL	Memory Fence Stalls	The event occurs during stalls for Memory Fence instruction.
	Memory Fence Stalls Per Cycle	Memory fence stalls, relative to number of cycles.
SLOW_SPR_STALL	Slow SPR Stalls	The event occurs during stalls to slow SPR access.
	Slow SPR Stalls Per Cycle	Slow SPR stalls, relative to number of cycles.
NETWORK_DEST_STALL	Network Blocked Stalls	The event occurs when a valid instruction in pipeline Decode stage stalls due to network destination being full.
	Network Blocked Stalls Per Cycle	Network blocked stalls, relative to number of cycles.

A.3 Memory Cache Statistics Events

Table 3. Memory Cache Statistics Events

Hardware Event	IDE Profile View Statistic	Description
ITLB_MISS_INTERRUPT	ITLB Miss Interupts	The event occurs when an ITLB_MISS interrupt is taken.
	ITLB Miss Interrupts Per Cycle	ITLB miss interrupts, relative to number of cycles.
READ	Reads	A load is issued.
READ_MISS	Read Misses	A load is issued and data is not returned from the level 1 data cache.
	Read Miss Ratio	Read misses, relative to number of reads.
	Read Hits	A load is issued and data is returned from the level 1 data cache.
	Read Hit Ratio	Read misses, relative to number of reads.
WRITE	Writes	A store is issued.
WRITE_MISS	Write Misses	A store is issued and the 16-byte aligned block (level 1 data cache line size) containing the address is not present at the level 1 data cache.
	Write Miss Ratio	Write misses, relative to number of writes.
	Write Hits	A store is issued and the 16-byte aligned block (level 1 data cache line size) containing the address is present at the level 1 data cache.
	Write Hit Ratio	Write hits, relative to number of writes.
TLB	TLB Access	A data memory operation is issued and the data translation lookaside buffer (DTLB) is used to translate the virtual address into the physical address.
	TLB Access Ratio	TLB accesses, relative to number of cycles.
TLB_EXCEPTION	TLB Exception	A data memory operation is issued and the address causes a Data TLB Exception including TLB Misses and protection violations.
	TLB Exception Ratio	TLB exceptions, relative to number of cycles.

Table 3. Memory Cache Statistics Events (continued)

Hardware Event	IDE Profile View Statistic	Description
LOCAL_INSTRUCTION_READ LOCAL_INSTRUCTION_READ_MISS	Local Instruction Reads	An instruction read request is received from the main processor and the Level 3 cache resides in the tile.
	Local Instruction Read Misses	An instruction read request is received from the main processor and misses the Level 3 cache in the tile.
	Local Instruction Read Miss Ratio	Local instruction read misses, relative to number of reads.
	Local Instruction Read Hits	A data read request is received from the main processor and hits the Level 3 cache in the tile.
	Local Instruction Read Hit Ratio	Local data read hits, relative to number of reads.
LOCAL_DATA_READ LOCAL_DATA_READ_MISS	Local Data Reads	A data read request is received from the main processor and the Level 3 cache resides in the tile.
	Local Data Read Misses	A data read request is received from the main processor and misses the Level 3 cache in the tile.
	Local Data Read Miss Ratio	Local data read misses, relative to number of reads.
	Local Data Read Hits	A data read request is received from the main processor and hits the Level 3 cache in the tile.
	Local Data Read Hit Ratio	Local data read hits, relative to number of reads.
LOCAL_WRITE LOCAL_WRITE_MISS	Local Data Writes	A write request is received from the main processor and the Level 3 cache resides in the tile.
	Local Data Write Misses	A write request is received from the main processor and misses the Level 3 cache resided in the tile.
	Local Data Write Miss Ratio	Local data write misses, relative to number of cycles.
	Local Data Write Hits	A write request is received from the main processor and hits the Level 3 cache in the tile.
	Local Data Write Hit Ratio	Local data write hits, relative to number of cycles.

Table 3. Memory Cache Statistics Events (continued)

Hardware Event	IDE Profile View Statistic	Description
REMOTE_INSTRUCTION_READ REMOTE_INSTRUCTION_READ_MISS	Remote Instruction Reads	An instruction read request is received from the main processor and the Level 3 cache resides in another tile.
	Remote Instruction Read Miss	An instruction read request is received from the main processor and misses the Level 2 cache. The Level 3 cache resides in another tile.
	Remote Instruction Read Miss Ratio	Remote instruction read misses, relative to number of reads.
	Remote Instruction Read Hits	An instruction read request is received from the main processor and hits the Level 2 cache. The Level 3 cache resides in another tile.
	Remote Instruction Read Hit Ratio	Remote instruction read hits, relative to number of reads.
REMOTE_DATA_READ REMOTE_DATA_READ_MISS	Remote Data Reads	A data read request is received from the main processor and the Level 3 cache resides in another tile.
	Remote Data Read Miss	A data read request is received from the main processor and misses the Level 2 cache. The Level 3 cache resides in another tile.
	Remote Data Read Miss Ratio	Remote data read misses, relative to number of cycles.
	Remote Data Read Hit	A data read request is received from the main processor and hits the Level 2 cache. The Level 3 cache resides in another tile.
	Remote Data Read Hit Ratio	Remote data read hits, relative to number of cycles.
REMOTE_WRITE REMOTE_WRITE_MISS	Remote Writes	A write request is received from the main processor and the Level 3 cache resides in another tile.
	Remote Write Misses	A write request is received from the main processor and misses the Level 2 cache. The Level 3 cache resides in another tile.
	Remote Write Miss Ratio	Remote write misses, relative to number of writes.
	Remote Write Hits	A write request is received from the main processor and hits the Level 2 cache. The Level 3 cache resides in another tile.
	Remote Write Hit Ratio	Remote write hits, relative to number of cycles.

Table 3. Memory Cache Statistics Events (continued)

Hardware Event	IDE Profile View Statistic	Description
SNP_WRITE SNP_WRITE_MISS	Snoop Writes	A write request is received from another tile off the SDN.
	Snoop Write Misses	A write request is received from another tile off the SDN and misses the Level 3 cache.
	Snoop Write Miss Ratio	Snoop write misses, relative to number of writes.
	Snoop Write Hits	A write request is received from another tile off the SDN and hits the Level 3 cache.
	Snoop Write Hit Ratio	Snoop write hits, relative to number of cycles.
SNOOP_INCREMENT_READ SNOOP_INCREMENT_READ_MISS	Snoop Increment Reads	A read request is received from another tile off the SDN and the Level 3 cache will track the share state.
	Snoop Increment Read Misses	A read request is received from another tile off the SDN and misses the Level 3 cache. The Level 3 cache will not track the share state.
	Snoop Increment Read Miss Ratio	Snoop non-incrementing read misses, relative to number of reads.
	Snoop Increment Read Hits	A read request is received from another tile off the SDN and hits the Level 3 cache. The Level 3 cache will not track the share state.
	Snoop Increment Read Hit Ratio	Snoop incrementing read hits, relative to number of reads.
SNOOP_NON_INCREMENT_READ SNOOP_NON_INCREMENT_READ_MISS	Snoop Non-Increment Reads	A read request is received from another tile off the SDN and the Level 3 cache will not track the share state.
	Snoop Non-Increment Read Misses	A read request is received from another tile off the SDN and misses the Level 3 cache. The Level 3 cache will not track the share state.
	Snoop Non-Increment Read Miss Ratio	Snoop non-incrementing read misses, relative to number of reads.
	Snoop Non-Increment Read Hits	A read request is received from another tile off the SDN and hits the Level 3 cache. The Level 3 cache will not track the share state.
	Snoop Non-Increment Read Hit Ratio	Snoop non-incrementing read hits, relative to number of reads.

Table 3. Memory Cache Statistics Events (continued)

Hardware Event	IDE Profile View Statistic	Description
SNOOP_IO_READ SNOOP_IO_READ_MISS	Snoop I/O Reads	A read request is received from an I/O device off the SDN.
	Snoop I/O Read Misses	A read request is received from an IO device off the SDN and misses the Level 3 cache.
	Snoop I/O Read Miss Ratio	Snoop I/O read misses, relative to number of writes.
	Snoop I/O Read Hits	A read request is received from an I/O device off the SDN and hits the Level 3 cache.
	Snoop I/O Read Hit Ratio	Snoop I/O read hits, relative to number of cycles
SNOOP_IO_WRITE SNOOP_IO_WRITE_MISS	Snoop I/O Writes	A write request is received from an I/O device off the SDN.
	Snoop I/O Write Misses	A write request is received from an I/O device off the SDN and misses the Level 3 cache.
	Snoop I/O Write Miss Ratio	Snoop I/O write misses, relative to number of writes.
	Snoop I/O Write Hits	A write request is received from an I/O device off the SDN and hits the Level 3 cache.
	Snoop I/O Write Hit Ratio	Snoop I/O write hits, relative to number of cycles.
COHERENCE_INVALIDATION COHERENCE_INVALIDATION_HIT	Coherence Invalidation	A coherence invalidation is received from another tile off the QDN.
	Coherence Invalidation Cache Hits	A coherence invalidation is received from another tile off the QDN and hits the level 2 cache.
	Coherence Invalidation Cache Hit Ratio	Coherence invalidation cache hits, relative to number of cycles.
	Coherence Invalidation Cache Misses	A coherence invalidation is received from another tile off the QDN and misss the level 2 cache.
	Coherence Invalidation Cache Miss Ratio	Coherence invalidation cache misss, relative to number of cycles.
CACHE_WRITEBACK	Cache Writeback	A cache writeback to main memory, including victim writes or explicit flushes, leaves the tile.
	Cache Writeback Ratio	Cache writebacks, relative to number of cycles.
SDN_STARVED	SDN Starved	A snoop is received and the controller enters the SDN starved condition.
	SDN Starved Ratio	SDN Starved events, relative to number of cycles.

Table 3. Memory Cache Statistics Events (continued)

Hardware Event	IDE Profile View Statistic	Description
RDN_STARVED	RDN Starved	A snoop is received and the controller enters the RDN starved condition.
	RDN Starved Ratio	RDN Starved events, relative to number of cycles.
QDN_STARVED	QDN Starved	A snoop is received and the controller enters the QDN starved condition.
	QDN Starved Ratio	QDN Starved events, relative to number of cycles.
SKF_STARVED	SKF Starved	A snoop is received and the controller enters the skid FIFO starved condition.
	SKF Starved Ratio	SKF Starved events, relative to number of cycles.
RTF_STARVED	RTF Starved	The event occurs when the controller enters the re-try FIFO starved condition.
	RTF Starved Ratio	RTF Starved events, relative to number of cycles.
IREQ_STARVED	IREQ Starved	The event occurs when the controller enters the instruction stream starved condition.
	IREQ Starved Ratio	IREQ Starved events, relative to number of cycles.
ITLB_OLOC_CACHE_MISS	Inst TLB OLOC Cache Miss	An instruction read request associated with the ITLB entry specified by ITLB_PERF is received from the main processor and misses the Level 2 cache. The Level 3 cache resides in another tile.
	Inst TLB OLOC Cache Miss Ratio	Instruction TLB OLOC cache misses, relative to number of cycles.
DTLB_OLOC_CACHE_MISS	Data TLB OLOC Cache Miss	A data read or write (load or store) request associated with the DTLB entry specified by DTLB_PERF is received from the main processor and misses the Level 2 cache. The Level 3 cache resides in another tile.
	Data TLB OLOC Cache Miss Ratio	Data TLB OLOC cache misses, relative to number of cycles.
LOCAL_WRITE_BUFFER_ALLOC	Local Write Buffer Alloc	A write request is received from the main processor and allocates a write buffer in the Level 3 cache resided in the tile.
	Local Write Buffer Alloc Ratio	Local write buffer hits, relative to number of cycles.
REMOTE_WRITE_BUFFER_ALLOC	Remote Write Buffer Alloc	A write request is received from the main processor and allocates a write buffer in the Level 2 cache resided in the tile. The Level 3 cache resides in another tile.
	Remote Write Buffer Alloc Ratio	Remote write buffer hits, relative to number of cycles.

A.4 iMesh Network Traffic Statistics Events

These events relate to the five dynamic networks comprising the interconnect fabric of the Tilera iMesh:

- UDN — User Dynamic Network
- IDN — Internal Dynamic Network
- QDN — reQuest Dynamic Network
- RDN — Response Dynamic Network
- SDN — Share Dynamic Network

Table 4. iMesh Network Traffic Statistics Events

Hardware Event	IDE Profile View Statistic	Description
UDN_PACKET_SENT	UDN Packet Sent	Main processor finished sending a packet to the UDN.
	UDN Packet Sent Ratio	UDN packets sent, relative to number of cycles.
UDN_WORD_SENT	UDN Word Sent	UDN word sent to an output port. Participating ports are selected with the UDN_EVT_PORT_SEL field.
	UDN Word Sent Ratio	UDN words sent, relative to number of cycles.
UDN_BUBBLE	UDN Bubble	Bubble detected on an output port. A bubble is defined as a cycle in which no data is being sent, but the first word of a packet has already traversed the switch. Participating ports are selected with the UDN_EVT_PORT_SEL field.
	UDN Bubble Ratio	UDN bubbles, relative to number of cycles.
UDN_CONGESTION	UDN Congestion	Out of credit on an output port. Participating ports are selected with the UDN_EVT_PORT_SEL field.
	UDN Congestion Ratio	UDN congestion events, relative to number of cycles.
UDN_DEMUX_STALL	UDN Demux Stall	UDN Demux stalled due to buffer full.
	UDN Demux Stall Ratio	UDN Demux stalls, relative to number of cycles.
IDN_PACKET_SENT	IDN Packet Sent	Main processor finished sending a packet to the IDN.
	IDN Packet Sent Ratio	IDN packets sent, relative to number of cycles.
IDN_WORD_SENT	IDN Word Sent	IDN word sent to an output port. Participating ports are selected with the IDN_EVT_PORT_SEL field.
	IDN Word Sent Ratio	IDN words sent, relative to number of cycles.
IDN_BUBBLE	IDN Bubble	Bubble detected on an output port. A bubble is defined as a cycle in which no data is being sent, but the first word of a packet has already traversed the switch. Participating ports are selected with the IDN_EVT_PORT_SEL field.
	IDN Bubble Ratio	IDN bubble events, relative to number of cycles.
IDN_CONGESTION	IDN Congestion	Out of credit on an output port. Participating ports are selected with the IDN_EVT_PORT_SEL field.
	IDN Congestion Ratio	IDN congestion event, relative to number of cycles.
IDN_DEMUX_STALL	IDN Demux Stall	IDN Demux stalled due to buffer full.
	IDN Demux Stall Ratio	IDN Demux stalls, relative to number of cycles.

Table 4. iMesh Network Traffic Statistics Events (continued)

Hardware Event	IDE Profile View Statistic	Description
RDN_PACKET_SENT	RDN Packet Sent	Main processor finished sending a packet to the RDN.
	RDN Packet Sent Ratio	RDN packets sent, relative to number of cycles.
RDN_WORD_SENT	RDN Word Sent	RDN word sent to an output port. Participating ports are selected with the RDN_EVT_PORT_SEL field.
	RDN Word Sent Ratio	RDN words sent, relative to number of cycles.
RDN_BUBBLE	RDN Bubble	Bubble detected on an output port. A bubble is defined as a cycle in which no data is being sent, but the first word of a packet has already traversed the switch. Participating ports are selected with the RDN_EVT_PORT_SEL field.
	RDN Bubble Ratio	RDN bubble events, relative to number of cycles,
RDN_CONGESTION	RDN Congestion	Out of credit on an output port. Participating ports are selected with the RDN_EVT_PORT_SEL field.
	RDN Congestion Ratio	RDN congestion events, relative to number of cycles.
SDN_PACKET_SENT	SDN Packet Sent	Main processor finished sending a packet to the SDN.
	SDN Packet Sent Ratio	SDN packets sent, relative to number of cycles.
SDN_WORD_SENT	SDN Word Sent	SDN word sent to an output port. Participating ports are selected with the SDN_EVT_PORT_SEL field,
	SDN Word Sent Ratio	SDN words sent, relative to number of cycles.
SDN_BUBBLE	SDN Bubble	Bubble detected on an output port. A bubble is defined as a cycle in which no data is being sent, but the first word of a packet has already traversed the switch. Participating ports are selected with the SDN_EVT_PORT_SEL field.
	SDN Bubble Ratio	SDN bubble events, relative to number of cycles.
SDN_CONGESTION	SDN Congestion	Out of credit on an output port. Participating ports are selected with the SDN_EVT_PORT_SEL field.
	SDN Congestion Ratio	SDN congestion events, relative to number of cycles.
QDN_PACKET_SENT	QDN Packet Sent	Main processor finished sending a packet to the QDN.
	QDN Packet Sent Ratio	QDN packets sent, relative to number of cycles.
QDN_WORD_SENT	QDN Sent	QDN word sent to an output port. Participating ports are selected with the QDN_EVT_PORT_SEL field.
	QDN Sent Ratio	QDN words sent, relative to number of cycles.
QDN_BUBBLE	QDN Bubble	Bubble detected on an output port. A bubble is defined as a cycle in which no data is being sent, but the first word of a packet has already traversed the switch. Participating ports are selected with the QDN_EVT_PORT_SEL field.
	QDN Bubble Ratio	QDN bubble events, relative to number of cycles.
QDN_CONGESTION	QDN Congestion	Out of credit on an output port. Participating ports are selected with the QDN_EVT_PORT_SEL field.
	QDN Congestion Ratio	QDN congestion events, relative to number of cycles.

APPENDIX B—SUGGESTED READING

John Bentley and Patrick Chan. *Programming Pearls*, Addison-Wesley Professional (Part of the ACM Press series), 1999.

David Culler, J.P. Singh, and Anoop Gupta. *Parallel Computer Architecture: A Hardware/Software Approach (The Morgan Kaufmann Series in Computer Architecture and Design)*. Morgan Kaufmann Publishers, Inc. 340 Pine Street, Sixth Floor, San Francisco, CA 94104-3205, 1999.

Joseph A. Fisher, Paolo Faraboschi, and Cliff Young. *Embedded Computing: A VLIW Approach to Architecture, Compilers and Tools*. Morgan Kaufmann Publishers (an imprint of Elsevier, 500 Sansome Street, Suite 400, San Francisco, CA 94111, 2005.

Christopher Hallinan. *Embedded Linux Primer: A Practical Real-World Approach*. Prentice Hall Open Source Software Development Series, Upper Saddle River, NJ, 2007.

Jain, Raj. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. John Wiley & Sons, Inc., New York, NY, 1991.

Donald E. Knuth. *Art of Computer Programming, The, Volumes 1-3 Boxed Set (2nd Edition) (The Art of Computer Programming Series)*, Addison-Wesley, 1998.

Scott J. Norton and Mark D. Dipasquale. *Thread Time: The MultiThreaded Programming Guide*, Hewett-Packard Professional Books, 1997.

Rush D. Robinett III, David G. Wilson, G. Richard Eisler, John E. Hurtado. *Applied Dynamic Programming for Optimization of Dynamical Systems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2005.

Henry S. Warren. *Hacker's Delight*, Pearson Education, 2002.

There is a good paper describing the gains from Link-time Optimization on spec benchmarks on x86. Even though the performance gain is often modest, this paper notes that program size often decreases drastically. http://arxiv.org/pdf/1010.2196v2

INDEX